

“Zivora – Shopping Cart and Checkout Flow”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING

&

DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING

By

K. Kushal Kumar 2520040031

T. Sai Pavan Reddy 2520040018

Under the Esteemed Guidance of

DR KOTTE ANUSHA

Assistant Professor

Department of Electronics and Communication Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING

&

DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING



Declaration

The Project Report entitled **“Shopping Cart and Checkout Flow”** is a record of Bonafide work of **K. Kushal Kumar - 2520040031, T. Sai Pavan Reddy - 2520040018** submitted in partial fulfillment for the award of B. Tech in Electronic Communication and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

K. KUSHAL KUMAR – 2520040031

T. SAI PAVAN REDDY – 2520040018

K L (Deemed to be) University

DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING

&

DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **“Shopping Cart and Checkout Flow”** is a bonafide work done and submitted by **K. Kushal Kumar - 2520040031, T. Sai Pavan Reddy - 2520040018** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Electronic Communication And Engineering, K L (Deemed to be University), during the academic year **2025-2026**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. K. MADHAVI REDDY**, and my guide **Dr. K. ANUSHA** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Sunkara Srinivasa Rao**, Head of the *Department for Electronics and Communication Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

K. KUSHAL KUMAR – 2520040031

T. SAI PAVAN REDDY – 2520040018

Abstract

Online shopping applications often struggle with inefficient checkout processes, including poor cart management, complicated navigation, and multiple unorganized steps. These issues can lead to user frustration and increased cart abandonment rates. The problem lies in the lack of a simple, structured, and user-friendly flow that guides customers smoothly from selecting items to completing a purchase. This project focuses on addressing these challenges by designing a clear and organized multi-step checkout process. The aim is to improve usability by simplifying interactions, reducing confusion, and ensuring a seamless transition between different stages of checkout. The solution emphasizes a better user experience, helping users complete their purchases quickly and efficiently.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	4
		Abstract	6
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	9-16
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	17-23
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	24-32
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	33-37
5	Design	ER Diagram	38
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	39-50
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	51-54
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	55-64
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	65-72
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	73-79
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	80-83
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	84-90
13	References	- Books, tutorials, APIs, documentation sites used	91-92

14	Appendices	- Screenshots of the Website - Sample code snippets - Installation/setup instructions - User manual or guide - Geo Tag photos with guide - Poster	93-97 98-100 101 101-102 103 104
----	------------	--	---

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

The rapid advancement of Information and Communication Technology (ICT) has transformed the way people perform everyday activities, including shopping. With the widespread availability of the Internet and smartphones, online shopping has become one of the most popular methods of purchasing products. Customers can now browse products, compare prices, place orders, and receive deliveries without visiting physical stores. This shift towards digital shopping has increased the demand for secure, fast, and user-friendly e-commerce applications.

Traditional shopping methods require customers to travel to stores, search for products, compare prices manually, and wait in billing queues. These processes consume time and effort, especially during weekends and festive seasons. Although many online shopping platforms are available today, some suffer from slow performance, complicated navigation, limited responsiveness, and poor user experience. These limitations highlight the need for a modern online shopping system that is simple, responsive, and easy to maintain.

To address these challenges, **Zivora – Online Shopping Platform** has been developed as a web-based e-commerce application that provides customers with a convenient and efficient shopping experience. The application allows users to register, log in securely, browse products, search by category, add items to the shopping cart, place orders, and manage their profiles through an intuitive interface. A separate administrator module enables authorized users to monitor customer information and manage application data efficiently.

The project has been developed using modern web technologies such as **React.js**, **Vite**, **React Router DOM**, **HTML5**, **CSS3**, and **JavaScript (ES6)** for the frontend. The backend is implemented using **Node.js** and **Express.js**, while a **JSON Database (db.json)** is used to store customer details, product information, shopping cart data, and order records. Communication between the frontend and backend is achieved through RESTful APIs, ensuring smooth and efficient data exchange.

One of the major strengths of Zivora is its **component-based and modular architecture**. Each feature, including Authentication, Product Management, Shopping Cart, Order Management, User Profile, and Administrator Dashboard, is implemented as an independent module. This approach improves code reusability, simplifies maintenance, and allows future enhancements without affecting the existing system. The application also follows responsive web design principles, ensuring smooth performance on desktops, laptops, tablets, and smartphones.

The project has been designed with scalability in mind. Although it currently uses a lightweight JSON database suitable for academic purposes, it can be upgraded in the future by integrating databases such as MongoDB or MySQL. Additional features like online payment gateways, AI-based product

recommendations, customer reviews, inventory management, and cloud deployment can also be incorporated with minimal modifications.

In conclusion, **Zivora – Online Shopping Platform** demonstrates the practical implementation of modern full-stack web development technologies to build a secure, responsive, and user-friendly e-commerce application. The project not only provides a better shopping experience for users but also serves as a valuable learning platform for understanding frontend development, backend programming, RESTful APIs, database management, and software engineering principles.

1.2 Problem Statement

The rapid growth of the Internet and digital technologies has transformed the retail industry by introducing online shopping platforms that allow customers to purchase products conveniently from anywhere and at any time. With the increasing use of smartphones and high-speed internet, customers expect e-commerce applications to provide a fast, secure, responsive, and user-friendly shopping experience. However, despite the availability of several online shopping websites, many existing systems still face limitations that affect customer satisfaction and overall shopping efficiency.

Traditional shopping methods require customers to physically visit stores, spend considerable time searching for products, compare prices manually, and wait in long billing queues before completing their purchases. This process is not only time-consuming but also inconvenient, especially during weekends, holidays, and festive seasons when stores experience heavy crowds. In addition, customers are restricted by store operating hours and geographical location, making shopping less flexible and accessible.

Although many online shopping platforms have been introduced to overcome these issues, several existing systems still suffer from problems such as slow loading speeds, complex navigation, poor mobile responsiveness, and inefficient product organization. Customers often experience difficulty in locating products quickly, managing shopping carts, and completing purchases smoothly. Some websites require multiple page reloads, which interrupts the shopping experience and increases response time. These shortcomings reduce customer satisfaction and negatively impact the overall usability of the application.

Another major challenge is the lack of a modular and scalable architecture in some existing shopping systems. Applications developed using tightly coupled components are difficult to maintain and extend. Integrating new features such as secure payment gateways, personalized product recommendations, inventory management, customer reviews, and real-time order tracking often requires significant modifications to the existing system. This increases software complexity, development time, and maintenance costs.

Security and efficient user management are also important concerns in modern e-commerce applications. Many systems do not provide proper role-based authentication to separate customer and administrator functionalities. In addition, maintaining product information, customer records, shopping carts, and order details efficiently requires a well-structured backend and database architecture. Without these features, managing application data becomes difficult and less reliable.

To overcome these challenges, Zivora – Online Shopping Platform has been developed as a modern, responsive, and user-friendly web application. The system provides secure user registration and login, organized product categories, dynamic shopping cart management, order processing, profile management, and a dedicated administrator dashboard. The application is developed using React.js, Vite, Node.js, Express.js, and a JSON Database, ensuring efficient communication between the

frontend and backend through RESTful APIs. The modular architecture improves maintainability, scalability, and future enhancements while providing customers with a smooth and efficient online shopping experience.

The proposed Zivora system aims to simplify the online shopping process, improve customer convenience, reduce shopping time, and provide a reliable platform for both customers and administrators. By integrating modern web technologies with an intuitive interface and efficient backend processing, the system enhances overall shopping efficiency and delivers a better digital shopping experience.

Problems Identified:

The major problems identified in the existing shopping systems are:

- Customers need to spend considerable time searching for products.
- Traditional shopping requires visiting physical stores.
- Long billing queues increase waiting time.
- Limited shopping hours reduce customer convenience.
- Slow loading speeds in some existing online shopping websites.
- Complex and confusing user interfaces.
- Poor responsiveness on mobile devices.
- Difficulty in managing shopping carts efficiently.
- Lack of secure role-based authentication for customers and administrators.
- Limited scalability and difficult maintenance due to non-modular architectures.
- Challenges in integrating advanced features such as payment gateways and personalized recommendations.
- Inefficient product organization and navigation in some existing systems.

The proposed Zivora – Online Shopping Platform is designed to overcome these limitations by providing a secure, responsive, scalable, and user-friendly e-commerce solution that simplifies product browsing, shopping cart management, order processing, and administrative operations while enhancing the overall online shopping experience.

1.3 Scope of the Project

The scope of the **Zivora – Online Shopping Platform** is to develop a modern, responsive, and user-friendly web application that enables customers to purchase products conveniently through an online platform. The project focuses on providing a simple and efficient shopping experience by integrating essential e-commerce functionalities such as user authentication, product browsing, shopping cart management, order processing, and profile management into a single application. The system is designed to simplify the shopping process while ensuring fast performance, secure user access, and an intuitive interface.

The application allows customers to create an account, log in securely, browse products across different categories, search for products, add products to the shopping cart, update cart items, place orders, and view their order history. These features provide customers with the flexibility to shop from any location using a desktop, laptop, tablet, or smartphone without the need to visit a physical store.

The project also includes an **Administrator Module** that provides authorized users with the ability to monitor customer information, manage product details, and oversee order records. By maintaining separate interfaces for

customers and administrators, the application ensures better security and organized system management through role-based access.

The frontend of the application has been developed using **React.js**, **Vite**, **React Router DOM**, **HTML5**, **CSS3**, and **JavaScript (ES6)** to provide a responsive Single Page Application (SPA). The backend is implemented using **Node.js** and **Express.js**, while a lightweight **JSON Database (db.json)** is used for storing customer information, product details, shopping cart records, and order information. Communication between the frontend and backend is achieved through RESTful APIs, enabling efficient data exchange and smooth system operation.

The current version of the project focuses on the core functionalities required for an online shopping platform. It demonstrates the practical implementation of modern full-stack web development technologies while providing a scalable architecture that supports future enhancements. The modular design of the application allows additional features to be integrated without significantly modifying the existing system.

Although the project currently uses a JSON database for simplicity and academic purposes, it can be upgraded to enterprise-level database systems such as **MongoDB**, **MySQL**, or **Firebase**. Future enhancements may also include secure online payment gateways, inventory management, customer reviews and ratings, wishlist functionality, AI-based product recommendations, real-time order tracking, email notifications, and cloud deployment. These enhancements will improve the functionality and scalability of the application while making it suitable for large-scale commercial use.

Thus, the scope of the **Zivora – Online Shopping Platform** extends beyond fulfilling the current project requirements by providing a strong foundation for developing a complete and scalable e-commerce system using modern web technologies.

Scope of the Project

The major scope of the proposed system includes:

- Development of a responsive and user-friendly online shopping platform.
- Secure customer registration and login with authentication.
- Product browsing through multiple categories such as Phones, Laptops, Watches, Bags, and Shoes.
- Product search and category-wise filtering.
- Shopping cart management with add, update, and remove functionalities.
- Order placement and order history management.
- User profile management.
- Separate administrator dashboard for system management.
- RESTful API communication between frontend and backend.
- Data storage using a JSON database.
- Responsive design compatible with desktops, laptops, tablets, and smartphones.
- Scalable architecture that supports future enhancements such as payment gateways, cloud databases, AI-based recommendations, inventory management, and real-time order tracking.

1.4 Objectives of the Project

The primary objective of the **Zivora – Online Shopping Platform** is to develop a secure, responsive, and user-friendly web application that simplifies the online shopping experience for customers while providing an efficient management system for administrators. The project aims to integrate modern frontend and backend technologies to build a scalable e-commerce platform capable of handling customer

registration, product management, shopping cart operations, and order processing efficiently.

The system has been designed to provide customers with a convenient shopping experience by allowing them to browse products, search by category, add items to the shopping cart, and place orders through an intuitive interface. The project also focuses on implementing secure authentication and role-based access control to ensure that customer and administrator functionalities remain separate and protected.

Another important objective is to demonstrate the practical implementation of modern Full-Stack Web Development technologies, including **React.js**, **Vite**, **Node.js**, **Express.js**, and a **JSON Database**. The project emphasizes modular software development, component reusability, RESTful API communication, and responsive web design, making the application easier to maintain and expand in the future.

Furthermore, the project aims to provide a strong foundation for integrating advanced e-commerce features such as secure payment gateways, cloud databases, AI-based product recommendations, inventory management, and real-time order tracking in future versions.

Objectives

The major objectives of the proposed system are:

- To develop a modern and responsive online shopping platform.
- To provide secure customer registration and login functionality.
- To implement role-based authentication for customers and administrators.
- To allow customers to browse products through different categories.
- To provide an efficient shopping cart for adding, updating, and removing products.
- To enable customers to place orders and view their order history.
- To develop an administrator dashboard for managing customers, products, and orders.
- To establish efficient communication between the frontend and backend using RESTful APIs.
- To store and manage application data using a JSON database.
- To develop a modular and scalable application that supports future enhancements.
- To ensure compatibility across desktops, laptops, tablets, and smartphones.
- To provide practical exposure to modern full-stack web development using React.js, Node.js, Express.js, and Vite.

1.5 Importance of the Application

The Zivora – Online Shopping Platform plays a significant role in providing a modern, efficient, and user-friendly solution for online shopping. As internet usage and digital commerce continue to grow

rapidly, customers expect shopping platforms that are fast, secure, and accessible from anywhere. Zivora addresses these expectations by offering a responsive web application that enables users to browse products, manage shopping carts, and place orders conveniently through a single platform. The application eliminates many of the limitations associated with traditional shopping methods, making the purchasing process simpler and more efficient.

One of the major importance of the application lies in its ability to save both time and effort for customers. Instead of visiting multiple physical stores to search for products, users can explore a wide range of items from different categories such as Mobile Phones, Laptops, Watches, Shoes, and Bags directly from their devices. The platform allows customers to compare products, view detailed information, and complete purchases without leaving their homes, making shopping more convenient and accessible.

The application also improves the overall shopping experience through its responsive and interactive user interface. Developed using React.js and Vite, the system provides smooth navigation between pages without unnecessary reloads. This enhances user satisfaction by delivering faster response times and a seamless browsing experience across desktops, laptops, tablets, and smartphones. The use of reusable React components also ensures consistency in the user interface while improving maintainability.

Another important aspect of Zivora is its secure user authentication and role-based access control. Customers can register and log in securely to access their accounts, while administrators have separate access to manage products, users, and orders. This separation of responsibilities improves application security and ensures efficient system administration.

The application also provides an effective shopping cart and order management system, allowing users to add products, modify quantities, remove unwanted items, and place orders easily. These features simplify the purchasing process while helping customers keep track of their selected products and previous orders. For administrators, the platform provides better control over product information and customer records, improving overall business management.

From an educational perspective, the project demonstrates the practical implementation of Full-Stack Web Development concepts. It combines frontend technologies such as React.js, HTML5, CSS3, JavaScript, and Vite with backend technologies including Node.js, Express.js, and a JSON Database. The project applies concepts such as component-based development, RESTful APIs, client-server architecture, routing, authentication, and database management, making it an excellent learning experience for students.

The modular architecture adopted in the application makes it highly scalable and easy to maintain. New features such as secure online payments, AI-based product recommendations, customer reviews, inventory management, wishlists, and cloud database integration can be added in future versions without significantly modifying the existing system. This flexibility makes Zivora suitable not only as an academic project but also as a foundation for developing a real-world e-commerce application.

Importance of the Application

The proposed Zivora – Online Shopping Platform is important because it:

- Provides customers with a fast and convenient online shopping experience.
- Eliminates the need to visit physical stores for purchasing products.
- Saves time and effort through digital product browsing and online ordering.
- Offers a responsive and user-friendly interface across multiple devices.

- Provides secure customer registration and authentication.
- Enables efficient shopping cart and order management.
- Supports administrators in managing products, users, and orders effectively.
- Improves software maintainability through a modular architecture.
- Demonstrates practical implementation of modern Full-Stack Web Development technologies.
- Provides a scalable platform that supports future enhancements such as online payment gateways, AI-based recommendations, cloud databases, inventory management, and real-time order tracking.

In conclusion, Zivora – Online Shopping Platform is an important web application that combines modern web technologies with essential e-commerce functionalities to deliver a secure, responsive, and efficient shopping experience. The application benefits both customers and administrators while serving as an excellent demonstration of contemporary software engineering principles and full-stack application development.

1.6 Target Users / Audience

The **Zivora – Online Shopping Platform** is designed to serve a wide range of users by providing a secure, convenient, and efficient online shopping experience. The application focuses on meeting the needs of customers while also providing administrators with the necessary tools to manage the platform effectively. The user-friendly interface, responsive design, and modular architecture make the application suitable for users with different levels of technical knowledge.

The primary users of the application are **customers** who wish to purchase products online. Customers can create an account, log in securely, browse products from different categories, add products to their shopping cart, place orders, and manage their profiles. The intuitive interface ensures that even first-time users can easily navigate the application without requiring technical assistance.

The application is also designed for **administrators**, who are responsible for managing the overall operation of the platform. Through the administrator dashboard, authorized users can monitor customer information, manage product details, oversee order records, and ensure the smooth functioning of the application. Role-based authentication ensures that administrative features remain secure and accessible only to authorized personnel.

In addition to customers and administrators, the project is valuable for **students, developers, and educational institutions**. It demonstrates the practical implementation of modern Full-Stack Web Development technologies such as React.js, Node.js, Express.js, and RESTful APIs. The project can be used as a learning resource for understanding component-based development, client-server architecture, routing, authentication, and database management.

The application has been developed using responsive web design techniques, allowing users to access the

platform through desktops, laptops, tablets, and smartphones. This flexibility ensures that users can shop conveniently from any location with an internet connection.

Target Users:

The major target users of the proposed system are:

- **Customers:** Browse products, add items to the shopping cart, place orders, and manage their accounts.
- **Administrators:** Manage products, customer information, and order details through the admin dashboard.
- **Small and Medium Businesses:** Use the platform as a foundation for developing and managing an online shopping website.
- **Students:** Learn modern Full-Stack Web Development concepts through a real-world e-commerce application.
- **Developers:** Study the implementation of React.js, Node.js, Express.js, RESTful APIs, and JSON-based data management.
- **Educational Institutions:** Use the project as a reference for academic learning, demonstrations, and software development practices.

The application is designed to provide an efficient, secure, and user-friendly shopping environment for all its users while demonstrating the practical application of modern web technologies. Its scalable architecture also makes it suitable for future enhancements, enabling it to evolve into a more comprehensive e-commerce solution as user requirements grow.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Hardware requirements refer to the physical components required for the successful development, testing, and execution of the **Zivora – Online Shopping Platform**. The selected hardware should support frontend development using **React.js**, backend development using **Node.js** and **Express.js**, and the execution of a **JSON Database**. A suitable hardware configuration ensures smooth application performance, faster development, and efficient testing.

The development system should have a modern processor capable of handling multiple development tools such as Visual Studio Code, Node.js, React development server, web browsers, and debugging utilities simultaneously. Adequate RAM is required to support multitasking, while sufficient storage space is necessary for source code, project dependencies, and application files. A stable internet connection is also essential for downloading packages, accessing online resources, and testing the application.

The application can be accessed using any desktop, laptop, tablet, or smartphone with a modern web browser and internet connection. Since the application is responsive, it performs efficiently across different screen sizes and devices.

Minimum Hardware Requirements

Processor

- Intel Core i3 Processor or Equivalent

Recommended

- Intel Core i5 / AMD Ryzen 5 or Higher

Memory (RAM)

- Minimum: 4 GB
- Recommended: 8 GB or Above

Storage

- Minimum: 20 GB Free Space
- Recommended: 50 GB SSD

Display

- Minimum Resolution: 1366 × 768 Pixels
- Recommended Resolution: 1920 × 1080 Pixels

Input Devices

- Keyboard

- Mouse / Touchpad

Internet

- Broadband Internet Connection

Hardware Configuration Table

Hardware Component	Specification
Processor	Intel Core i3 or Higher
RAM	4 GB Minimum
Storage	20 GB Free Space
Display	1366 × 768 Resolution
Input Devices	Keyboard & Mouse
Internet	Broadband Connection

The above hardware configuration is sufficient for the development, testing, and deployment of the **Zivora – Online Shopping Platform**. It provides a stable environment for running React.js, Node.js, Express.js, and the JSON database while ensuring smooth application performance.

2.2 Software Requirements

Software requirements refer to the software tools, frameworks, programming languages, and development environments required for designing, developing, testing, and deploying the **Zivora – Online Shopping Platform**. Selecting the appropriate software technologies is essential to ensure better performance, security, maintainability, and scalability of the application. The proposed system is developed using modern Full-Stack Web Development technologies that provide a responsive user interface, efficient backend processing, and reliable data management.

The frontend of the application is developed using **React.js**, a popular JavaScript library that enables developers to build reusable and interactive user interface components. **Vite** is used as the frontend build tool because it offers faster development, quick compilation, and Hot Module Replacement (HMR), making the development process more efficient.

The backend of the application is developed using **Node.js** and **Express.js**. Node.js provides a JavaScript runtime environment that allows server-side execution of JavaScript code, while Express.js simplifies backend development by providing routing, middleware, and RESTful API support. These technologies enable smooth communication between the frontend and backend.

The application stores its data using a **JSON Database (db.json)**, which is lightweight, easy to configure, and suitable for academic projects. It stores customer information, product details, shopping cart records, and order information. The development process is carried out using **Visual Studio Code**, while **Git** is used for version control and project management. The application is tested using modern web browsers such as Google Chrome, Microsoft Edge, and Mozilla Firefox to ensure compatibility and responsiveness across different platforms.

Software Specifications

Operating System:

The application can be developed and executed on the following operating systems:

- Windows 10 / Windows 11
- Linux
- macOS

Programming Languages:

- JavaScript (ES6)
- HTML5
- CSS3

Frontend Technologies:

- React.js
- Vite
- React Router DOM

Backend Technologies:

- Node.js
- Express.js

Database:

- JSON Database (db.json)

Development Tools:

- Visual Studio Code
- npm (Node Package Manager)
- Git

Web Browsers:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Software Configuration Table

Software Component	Specification
Operating System	Windows 10/11, Linux, macOS
Programming Language	JavaScript (ES6)
Frontend	React.js, HTML5, CSS3, Vite
Backend	Node.js, Express.js
Database	JSON Database (db.json)
IDE	Visual Studio Code
Package Manager	npm
Version Control	Git
Browser	Google Chrome, Microsoft Edge, Mozilla Firefox

Advantages of the Selected Software

The selected software technologies provide several advantages for the development of the **Zivora – Online Shopping Platform**:

- Supports modern Full-Stack Web Development.
 - Provides a fast and responsive user interface.
 - Enables reusable component-based architecture.
 - Facilitates efficient communication using RESTful APIs.
 - Simplifies backend development with Express.js.
 - Provides lightweight data management using JSON Database.
 - Ensures cross-platform compatibility.
 - Easy to maintain and extend with additional features.
 - Supports future integration with MongoDB, MySQL, payment gateways, and cloud services.
- The above software requirements provide a reliable environment for developing, testing, and deploying the **Zivora – Online Shopping Platform** while ensuring better performance, scalability, and maintainability.

2.3 Development Tools and Frameworks

Development tools and frameworks play an important role in the successful design, implementation, testing, and maintenance of the **Zivora – Online Shopping Platform**. These tools provide an efficient development environment, simplify coding tasks, improve productivity, and help developers build a responsive, scalable, and user-friendly web application.

The project is developed using modern Full-Stack Web Development technologies. The frontend is built with **React.js** and **Vite**, while the backend is developed using **Node.js** and **Express.js**. Additional tools such as **Visual Studio Code**, **npm**, **Git**, and **React Router DOM** are used to improve development efficiency and application performance.

React.js

React.js is the primary frontend library used for developing the **Zivora – Online Shopping Platform**. It is an open-source JavaScript library developed by Meta (Facebook) for building interactive and reusable user interfaces. React follows a **component-based architecture**, allowing developers to create reusable components such as Login, Registration, Product Cards, Shopping Cart, Orders, and Admin Dashboard.

Features of React.js

- Component-based architecture
- Virtual DOM for faster rendering
- Reusable UI components
- Efficient state management
- Improved application performance
- Easy maintenance and scalability

Vite

Vite is the frontend build tool used for developing and running the React application. It provides a fast development server, instant updates through **Hot Module Replacement (HMR)**, and optimized production builds. Vite improves development

speed and reduces application startup time.

Features of Vite

- Fast development server
 - Instant Hot Module Replacement (HMR)
 - Optimized production builds
 - Lightweight configuration
 - Faster application startup
-

Visual Studio Code

Visual Studio Code (VS Code) is the Integrated Development Environment (IDE) used for writing and managing the project source code. It provides features such as syntax highlighting, intelligent code completion, debugging support, an integrated terminal, and Git integration.

Features of Visual Studio Code

- Syntax highlighting
 - Auto-completion (IntelliSense)
 - Debugging support
 - Integrated terminal
 - Git integration
 - Extension support
-

Node.js

Node.js is the runtime environment used for executing JavaScript on the server side. It enables backend development, API creation, and package management using npm. In Zivora, Node.js is responsible for handling client requests and running the Express server.

Features of Node.js

- High-performance JavaScript runtime
 - Asynchronous and event-driven architecture
 - Fast server-side execution
 - Cross-platform support
 - Large npm ecosystem
-

Express.js

Express.js is a lightweight web framework built on Node.js. It simplifies backend development by providing routing, middleware support, and RESTful API creation. It handles communication between the frontend and the JSON database.

Features of Express.js

- Lightweight framework
 - REST API development
 - Routing support
 - Middleware integration
 - Easy backend management
-

React Router DOM

React Router DOM is used to implement client-side routing in the application. It enables smooth navigation between pages such as Home, Login, Register, Products, Cart, Orders, Profile, and Admin Dashboard without reloading the browser.

Features of React Router DOM

- Single Page Application (SPA) routing
 - Dynamic navigation
 - Protected routes
 - Fast page transitions
 - Improved user experience
-

Git

Git is used for version control during project development. It helps track source code changes, manage different versions of the project, and supports collaboration among team members.

Features of Git

- Version control
 - Code backup
 - Change tracking
 - Team collaboration
 - Easy project management
-

Benefits of the Development Tools

The selected development tools and frameworks provide several advantages:

- Faster application development.
- Component-based and reusable code structure.
- Smooth navigation using React Router DOM.
- Efficient backend development using Express.js.
- Easy debugging and testing.
- Better code maintainability and scalability.
- Faster development using Vite.
- Version control and project management with Git.
- Easy integration of future features such as payment gateways, cloud databases, and AI-based recommendations.

The combination of these modern development tools provides a reliable and efficient environment for building the **Zivora – Online Shopping Platform**. They ensure that the application is responsive, maintainable, scalable, and capable of supporting future enhancements while delivering a seamless shopping experience to users.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The frontend design of the **Zivora – Online Shopping Platform** focuses on providing a modern, responsive, and user-friendly interface that enables customers to browse products, manage shopping carts, and place orders efficiently. The frontend acts as the communication layer between users and the application by displaying product information, handling user interactions, and providing smooth navigation across different pages.

The application is developed using **React.js**, a modern JavaScript library that supports **component-based development**. This approach divides the application into reusable components such as **Home, Login, Register, Products, Cart, Orders, Profile, and Admin Dashboard**. Component-based architecture improves code reusability, maintainability, and scalability while making future enhancements easier.

The frontend is designed with simplicity and responsiveness in mind so that users can easily browse products, search by category, add products to the shopping cart, and complete purchases without technical difficulty. The user interface has been designed to provide an attractive shopping experience while ensuring compatibility across desktops, laptops, tablets, and smartphones.

The application uses **React Router DOM** for client-side routing, enabling smooth navigation between different pages without refreshing the browser. This Single Page Application (SPA) architecture improves performance and enhances the overall user experience. React Hooks such as **useState** and **useEffect** are used to manage application state and dynamically update user interface components.

The frontend consists of several modules that work together to provide complete shopping functionality.

Home Page Interface

The Home page serves as the entry point of the application. It displays featured products, product categories, and navigation links that allow users to explore the shopping platform easily.

Authentication Interface

The Authentication module allows users to register, log in securely, and access their accounts. It also provides separate login functionality for administrators.

Product Interface

The Product module displays products from different categories such as **Phones, Laptops, Watches, Bags, and Shoes**. Customers can browse products, view details, and select items for purchase.

Shopping Cart Interface

The Shopping Cart module allows customers to add products, update quantities, remove unwanted items, and calculate the total purchase amount before placing an order.

Order Management Interface

This module enables customers to place orders and view their previous order history. It simplifies the purchasing process by maintaining customer order records.

Profile Interface

The Profile module allows customers to view and manage their personal information and account details.

Administrator Dashboard

The Admin Dashboard enables authorized administrators to monitor registered customers and manage application data efficiently.

Frontend Design Principles

The frontend design follows the following principles:

- Simple and user-friendly interface.
 - Responsive web design.
 - Component-based architecture.
 - Reusable React components.
 - Fast page loading.
 - Smooth navigation using React Router DOM.
 - Easy accessibility across devices.
 - Consistent user interface.
-

Advantages of the Front-End Design

The frontend design provides several benefits:

- Attractive and responsive user interface.
- Easy product browsing and navigation.
- Improved customer shopping experience.
- Fast page transitions without reloading.
- Reusable and maintainable code structure.
- Better scalability for future enhancements.
- Cross-device compatibility.
- Improved application performance through React.js.

In conclusion, the frontend design of the **Zivora – Online Shopping Platform** provides a responsive, interactive, and efficient user interface that simplifies online shopping for customers while supporting administrators in managing the application. The use of **React.js**, **React Router DOM**, and modern frontend technologies ensures high performance, maintainability, and flexibility for future development.

3.2 Back-End Design

The Zivora – Online Shopping Platform is currently developed as a frontend-based web application using React.js. The present version of the project focuses on implementing product browsing, user authentication, shopping cart management, order placement, and administrative functionalities through client-side technologies. Therefore, a dedicated backend server and database management system have not been implemented in the current version of the application.

Unlike traditional e-commerce applications that rely on server-side processing, the current implementation performs all operations within the browser using React.js components, JavaScript, and Local Storage. User authentication, shopping cart data, product information, and order details are temporarily stored on the client side without requiring server communication.

Since the application is intended as a demonstration of an online shopping platform, advanced backend technologies such as Node.js, Express.js, PHP, Django, or Spring Boot have not been integrated. Similarly, database management systems such as MySQL, MongoDB, PostgreSQL, or Firebase Firestore are not used in the current implementation.

The absence of a backend simplifies the system architecture, reduces development complexity, and makes deployment easier because there is no need to configure application servers, APIs, or database services. All business logic and user interactions are handled directly within the browser, resulting in a lightweight and efficient application.

For demonstration purposes, product information, user credentials, shopping cart details, and order records are maintained using browser storage. This approach enables users to experience the core functionalities of an e-commerce platform without requiring an internet-connected backend infrastructure.

Current Back-End Functionality

Although no dedicated backend server is implemented, the application provides the following functionalities through client-side processing:

- User registration and login validation.
- Product information management.
- Shopping cart operations.
- Order placement simulation.
- Local storage of user and order data.
- Administrator login and dashboard access.
- Client-side routing and navigation.

Advantages of the Current Design

The current backend design offers several benefits:

- Simple and lightweight system architecture.
- Faster application development.
- Easy testing and debugging.
- No server maintenance required.
- No database configuration required.
- Easy deployment on frontend hosting platforms.
- Reduced development cost.

- Suitable for academic and demonstration purposes.

Limitations

The current implementation has the following limitations:

- No permanent cloud database.
- No real-time synchronization between users.
- Limited authentication security.
- No server-side validation.
- No online payment gateway integration.
- No inventory management.
- Data may be lost if browser storage is cleared.
- Not suitable for large-scale commercial deployment.

Future Enhancements

The backend can be enhanced in future versions by integrating:

- Node.js and Express.js for server-side development.
- MongoDB or MySQL for permanent database storage.
- JWT (JSON Web Token) for secure user authentication.
- REST APIs for frontend-backend communication.
- Cloudinary for product image storage.
- Razorpay or Stripe for online payment processing.
- Firebase Authentication for secure login.
- Cloud hosting using AWS, Vercel, Render, or Railway.

In Conclusion, The current version of the Zivora – Online Shopping Platform adopts a frontend-only architecture that successfully demonstrates the core features of an online shopping system using React.js and Local Storage. Although a dedicated backend is not implemented, the application effectively showcases product browsing, shopping cart management, user authentication, and order processing. The modular architecture also provides a strong foundation for integrating server-side technologies and cloud databases in future versions.

3.3 Database

The database is an essential component of any web application because it is responsible for storing, organizing, and retrieving application data. In the **Zivora – Online Shopping Platform**, a **JSON Database (db.json)** is used for storing product information, user details, shopping cart records, and order data. The JSON database is lightweight, easy to configure, and suitable for academic projects and prototype applications.

Unlike traditional relational databases such as MySQL or PostgreSQL, a JSON database stores data in the form of **JavaScript Object Notation (JSON)**. Each record is represented as key-value pairs, making the data easy to read, modify, and manage. The database is accessed through REST APIs provided by the backend, enabling smooth communication between the React frontend and the stored data.

The JSON database provides a simple and efficient way to perform **CRUD (Create, Read, Update, Delete)** operations. Customers can register, browse products, add items to the shopping cart, and place orders, while administrators can manage application data through the admin dashboard. Since the project is intended for

educational purposes, the JSON database provides sufficient functionality without requiring the complexity of a full-scale database management system.

The modular structure of the application allows the current database to be upgraded easily in the future. Enterprise databases such as **MongoDB**, **MySQL**, or **PostgreSQL** can replace the JSON database with minimal changes to the application architecture.

Database Features

- Lightweight and easy to configure.
- Stores data in JSON format.
- Supports CRUD (Create, Read, Update, Delete) operations.
- Easy integration with REST APIs.
- Fast data retrieval for small and medium-sized applications.
- Simple to maintain and modify.
- Suitable for academic projects and prototypes.

Data Stored in the Database

The JSON database stores the following information:

- User registration details.
- Login credentials.
- Product information.
- Product categories.
- Shopping cart data.
- Customer orders.
- Administrator information.

Advantages of JSON Database

- Simple structure and easy to understand.
- No complex database installation required.
- Faster development and testing.
- Lightweight and efficient for small applications.
- Easy integration with Node.js and Express.js.
- Supports future migration to MongoDB or MySQL.

Limitations

- Not suitable for very large-scale applications.
 - Limited support for concurrent users.
 - No advanced indexing or query optimization.
 - Less secure compared to enterprise database systems.
 - Limited scalability for commercial deployment.
-

Database Summary

Database Component	Specification
Database Type	JSON Database
File Name	db.json
Data Format	JSON (JavaScript Object Notation)
Operations	Create, Read, Update, Delete (CRUD)
Access Method	REST APIs
Suitable For	Academic Projects and Prototype Applications

Conclusion

The **JSON Database (db.json)** provides a simple, lightweight, and efficient solution for storing and managing the data used in the **Zivora – Online Shopping Platform**. It supports essential e-commerce operations such as user management, product storage, shopping cart handling, and order management. Although it is designed for academic use, the application's modular architecture allows easy migration to advanced database systems such as **MongoDB** or **MySQL** in future versions.

3.4 Version Control

Version control is an essential part of modern software development that helps developers manage source code, track changes, and collaborate efficiently. It maintains a history of modifications made to the project, allowing developers to restore previous versions whenever necessary. In the development of the **Zivora – Online Shopping Platform**, **Git** is used as the version control system, while **GitHub** serves as the remote repository for storing and managing the project source code.

Git is a distributed version control system that enables developers to record every change made to the project files. It allows multiple developers to work on different features simultaneously without affecting the main project. By using Git, developers can create branches, merge updates, resolve conflicts, and maintain different versions of the application throughout the development process.

GitHub is a cloud-based platform that hosts Git repositories and provides collaboration features such as repository management, issue tracking, code review, and project documentation. It also serves as a secure backup of the project and enables developers to access the source code from different systems.

Using Git and GitHub improved project organization, reduced the risk of data loss, and simplified collaboration during the development of the **Zivora – Online Shopping Platform**. These tools also support future development by maintaining a complete history of the project's evolution.

Features of Git

- Tracks changes made to source code.
 - Maintains complete version history.
 - Supports branching and merging.
 - Enables collaboration among developers.
 - Helps restore previous project versions.
 - Reduces the risk of data loss.
-

Features of GitHub

- Cloud-based repository hosting.
 - Online backup of project source code.
 - Easy collaboration among team members.
 - Repository management.
 - Issue tracking and documentation.
 - Secure access from multiple devices.
-

Benefits of Using Version Control

- Efficient management of project files.
 - Easy tracking of code modifications.
 - Simplifies debugging and error recovery.
 - Supports collaborative software development.
 - Prevents accidental loss of source code.
 - Facilitates future project enhancements.
 - Improves software maintenance.
-

Version Control Tools

Tool	Purpose
Git	Version control and source code management
GitHub	Cloud repository hosting and collaboration
Visual Studio Code	Git integration and code editing

Conclusion

Git and GitHub provide an efficient version control environment for the **Zivora – Online Shopping Platform**. They help manage source code changes, support collaboration, maintain project history, and ensure secure storage of the application. The use of version control improves software quality, simplifies maintenance, and provides a strong foundation for future development and deployment.

3.5 APIs or External services

Application Programming Interfaces (APIs) and external services enable different software components to communicate and exchange data efficiently. In the **Zivora – Online Shopping Platform**, APIs are used to establish communication between the React frontend and the backend developed using **Node.js** and **Express.js**. These APIs allow the application to perform various operations such as user authentication, product retrieval, shopping cart management, and order processing.

The project follows the **RESTful API** architecture, where the frontend sends HTTP requests to the backend, and the backend processes these requests before interacting with the **JSON Database (db.json)**. The backend returns the requested data in **JSON format**, which is then displayed dynamically on the user interface.

The current version of the project primarily uses **internal REST APIs** developed with Express.js. Since the application is designed for academic purposes, external payment gateways and third-party authentication services have not been integrated. However, the modular architecture allows such services to be added easily in future versions.

REST API

REST (Representational State Transfer) is the communication method used between the frontend and backend of the application. It enables efficient data exchange through standard HTTP methods.

HTTP Methods Used

- **GET** – Retrieve product and user information.
- **POST** – Register users and place orders.
- **PUT** – Update user profile and shopping cart.
- **DELETE** – Remove products from the shopping cart.

Services Provided by the APIs

The APIs are responsible for the following operations:

- User Registration
- User Login
- Product Retrieval
- Product Category Management
- Shopping Cart Operations
- Order Processing
- User Profile Management
- Administrator Operations

Future External Services

The following external services can be integrated in future versions of the project:

- **Razorpay** – Online payment gateway.
- **Stripe** – Secure online payment processing.
- **Firebase Authentication** – Secure user authentication.
- **Cloudinary** – Product image storage.
- **MongoDB Atlas** – Cloud database.
- **Email Notification Services** – Order confirmation and password recovery.

Advantages of Using APIs

- Enables smooth communication between frontend and backend.
 - Supports fast and efficient data transfer.
 - Simplifies application development.
 - Improves scalability and maintainability.
 - Allows easy integration of third-party services.
 - Supports future application enhancements.
-

API Summary Table

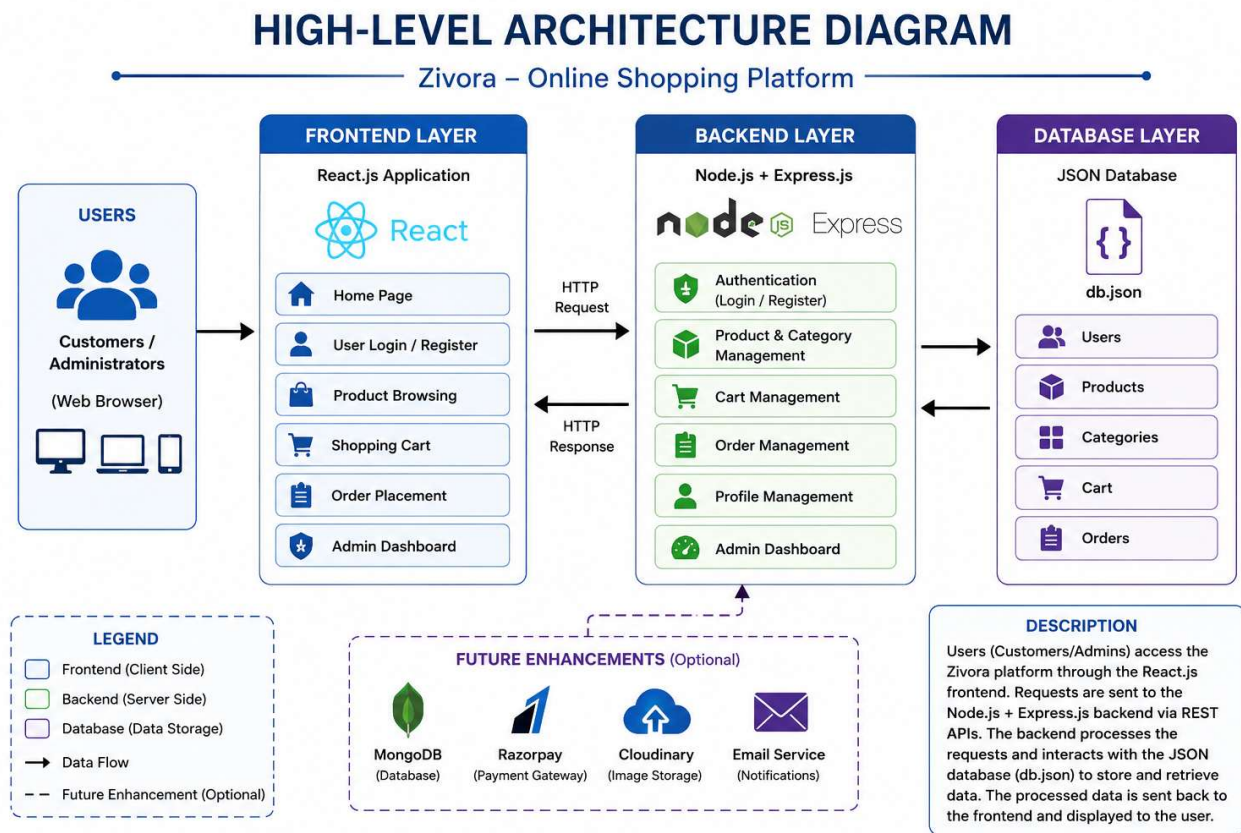
API / Service	Purpose
REST API	Communication between frontend and backend
Express.js API	Handles HTTP requests and responses
JSON Database	Stores application data
React Fetch API / Axios	Sends API requests from the frontend
Future Integration	Razorpay, Stripe, Firebase, Cloudinary

Conclusion

The **Zivora – Online Shopping Platform** uses **RESTful APIs** to establish efficient communication between the React frontend and the Express.js backend. These APIs enable smooth data exchange for user authentication, product management, shopping cart operations, and order processing. Although the current implementation uses a **JSON Database**, the application is designed to support future integration with external services such as payment gateways, cloud databases, authentication providers, and notification systems, making it scalable and suitable for future expansion.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram



4.2 Description of Each Layer

1. User Layer

The **User Layer** represents the end users who interact with the **Zivora – Online Shopping Platform**. The primary users include **customers** and **administrators**. Customers use the application to browse products, add items to the shopping cart, place orders, and manage their profiles. Administrators access the Admin Dashboard to manage products, categories, users, and customer orders. This layer acts as the starting point for all interactions within the system.

Responsibilities

- Access the application through a web browser.
- Register and log in securely.
- Browse products and categories.
- Search for products.
- Add and remove items from the shopping cart.
- Place orders.
- Manage user profiles.

- Access the Admin Dashboard (Administrator only).

2. Frontend Layer

The **Frontend Layer** is developed using **React.js**, **Vite**, **HTML5**, **CSS3**, **JavaScript**, and **React Router DOM**. It provides a responsive and interactive user interface where users can browse products, navigate between pages, and perform shopping operations. The application follows a component-based architecture, making the code reusable and easy to maintain.

Responsibilities

- Display the user interface.
- Handle user interactions.
- Render product information.
- Manage page navigation.
- Perform client-side validation.
- Communicate with the backend using REST APIs.
- Manage application state using React Hooks.
-

Main Modules

- Home Page
- Authentication (Login/Register)
- Product Browsing
- Product Categories
- Shopping Cart
- Order Placement
- Profile Management
- Admin Dashboard

3. Backend Layer

The **Backend Layer** is implemented using **Node.js** and **Express.js**. It processes requests received from the frontend, executes business logic, and communicates with the database. The backend exposes **RESTful APIs** that handle authentication, product management, shopping cart operations, and order processing.

Responsibilities

- Authenticate users.
- Manage products and categories.
- Process shopping cart requests.
- Handle order management.
- Manage user profiles.
- Provide administrator operations.
- Exchange data with the database through APIs.

4. Database Layer

The **Database Layer** uses a **JSON Database (db.json)** to store application data. It maintains user accounts, product details, product categories, shopping cart information, and order records. The backend performs CRUD (Create, Read, Update, Delete) operations on this data whenever requested by the application.

Responsibilities

- Store user information.
- Store product details.
- Maintain product categories.
- Store shopping cart data.

- Store order information.
- Support CRUD operations.
- Provide data to the backend.

Working of the Architecture

The user accesses the **Zivora – Online Shopping Platform** through a web browser. The React.js frontend displays the user interface and collects user inputs such as login requests, product searches, shopping cart updates, and order placements. These requests are sent to the **Node.js + Express.js** backend through RESTful APIs. The backend processes the requests, interacts with the **JSON Database (db.json)** to retrieve or update data, and returns the response to the frontend. Finally, the frontend displays the processed information to the user, providing a smooth and responsive shopping experience.

Figure Description

The architecture illustrates a layered design in which customers and administrators access the application through a web browser. The **React.js frontend** handles user interaction and communicates with the **Node.js + Express.js backend** using REST APIs. The backend performs the required business logic and stores or retrieves data from the **JSON Database (db.json)**. This layered architecture ensures modularity, scalability, easy maintenance, and efficient communication between different components of the **Zivora – Online Shopping Platform**.

4.3 Deployment Architecture

The deployment architecture of the **Zivora – Online Shopping Platform** describes how the application is executed and made accessible to users. The application follows a client-server architecture in which the frontend is developed using **React.js** and **Vite**, while the backend is implemented using **Node.js** and **Express.js**. The backend communicates with a **JSON Database (db.json)** to store and retrieve application data.

The frontend application is accessed through a modern web browser such as Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari. User requests are sent to the backend through RESTful APIs. The backend processes the requests, performs the required business logic, accesses the JSON database, and returns the appropriate response to the frontend.

The current implementation is primarily developed and tested in a local development environment. However, the application architecture supports future deployment on cloud platforms without significant modifications.

Local Deployment

The project is developed and tested locally using the Vite development server.

Deployment Steps

- Install Node.js and npm.
- Download the project source code.
- Install all project dependencies using npm.
- Start the Express backend server.
- Start the React application using the Vite development server.
- Open the application in a web browser using the local development URL.
-

This deployment method enables developers to test both frontend and backend modules efficiently before production deployment.

Cloud Deployment

Although the current version is executed locally, the application can be deployed to cloud platforms in future versions.

Possible deployment platforms include:

- Vercel (Frontend)
- Netlify (Frontend)
- Render (Backend)
- Railway (Backend)
- AWS (Amazon Web Services)
- Microsoft Azure
- Google Cloud Platform (GCP)
-

Cloud deployment will improve application availability, scalability, and accessibility for users.

CI/CD Pipeline

The current version of the project does not implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline. Development, testing, and deployment activities are performed manually.

Future versions may integrate:

- GitHub Actions
- Jenkins
- GitLab CI/CD
- Azure DevOps

These tools can automate code building, testing, and deployment, reduce manual effort and improving software quality.

Deployment Architecture Components

Component	Description
User Device	Desktop, Laptop, Tablet, or Mobile
Web Browser	Accesses the application
React.js Application	Provides the user interface
Vite	Frontend development server
Node.js	Runtime environment
Express.js	Backend server
REST APIs	Communication between frontend and backend
JSON Database (db.json)	Stores users, products, categories, cart, and orders

Advantages of the Deployment Architecture

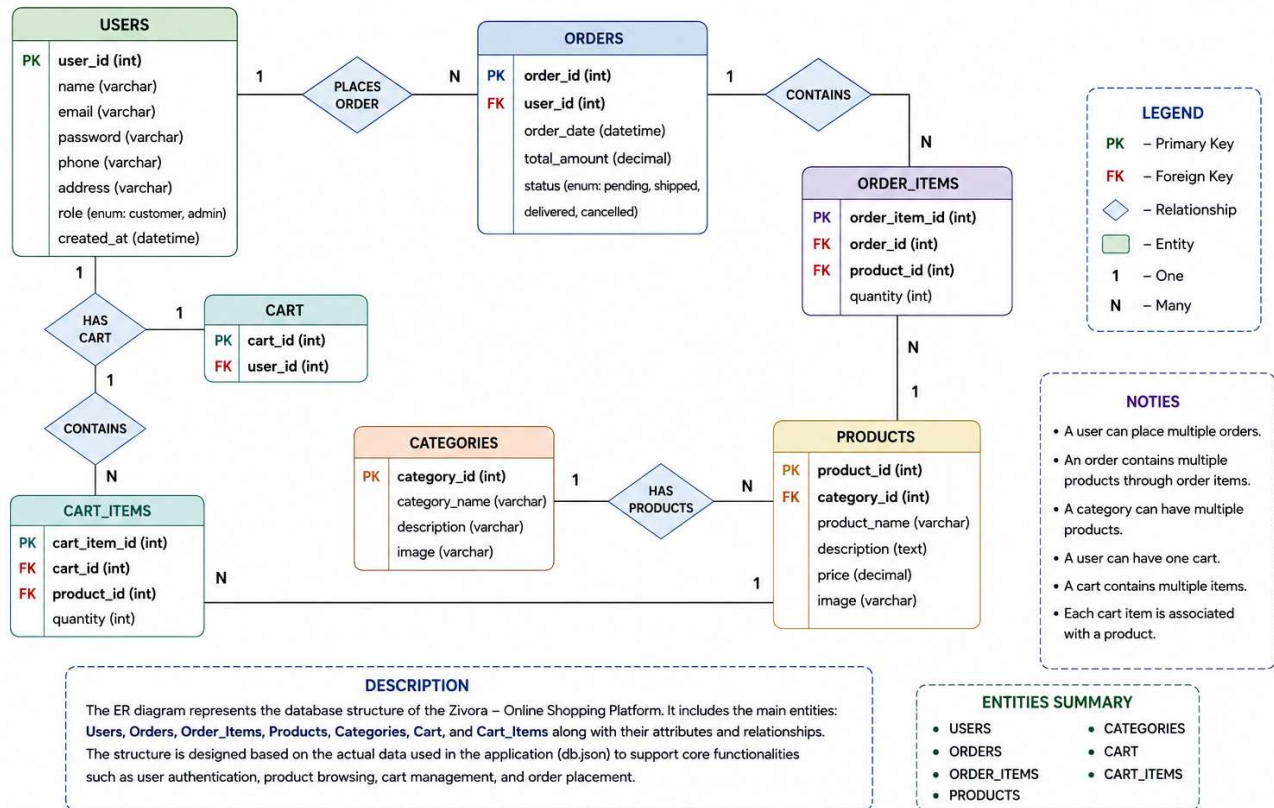
- Simple and modular architecture.
 - Easy development and maintenance.
 - Efficient communication through REST APIs.
 - Responsive user interface.
 - Lightweight database implementation.
 - Supports future cloud deployment.
 - Easy integration with advanced databases and payment gateways.
 - Scalable architecture for future enhancements.
-

Conclusion

The **Zivora – Online Shopping Platform** follows a client-server deployment architecture that combines a React.js frontend, a Node.js and Express.js backend, and a JSON database. The application is currently developed and tested in a local environment using the Vite development server and Express backend. The modular deployment architecture provides a stable foundation for future enhancements, including cloud hosting, enterprise databases, secure payment gateways, and automated CI/CD pipelines. It ensures efficient communication between different system components while maintaining scalability, maintainability, and ease of deployment.

CHAPTER -5 DESIGN

ER DIAGRAM FOR ZIVORA – ONLINE SHOPPING PLATFORM



CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

The Zivora – Online Shopping Platform is developed using a modular approach to improve code organization, maintainability, scalability, and reusability. Each module performs a specific function while interacting with other modules through well-defined interfaces. The application is developed using React.js, Vite, Node.js, Express.js, and a JSON Database (db.json). The major modules implemented in the system are described below.

1. User Authentication Module

The User Authentication Module manages user registration and login. It verifies user credentials and allows only authenticated users to access the application. Separate login functionality is provided for customers and administrators.

Functions:

- User Registration
 - User Login
 - Authentication Validation
 - Logout
 - Role-based Access (Customer/Admin)
-

2. Product Management Module

The Product Management Module allows users to browse products available in different categories. Each product contains details such as name, description, price, image, and category.

Functions:

- Display Products
 - View Product Details
 - Product Categorization
 - Product Search
 - Product Filtering
-

3. Category Management Module

This module organizes products into different categories, making it easier for users to browse and locate products.

Functions:

- Display Categories
 - Categorize Products
 - Category-wise Product Listing
-

4. Shopping Cart Module

The Shopping Cart Module enables customers to add products, update quantities, remove products, and view the total amount before placing an order.

Functions:

- Add to Cart
 - Update Quantity
 - Remove Products
 - Calculate Total Price
 - View Cart Summary
-

5. Order Management Module

The Order Management Module processes customer orders and maintains order information. It allows users to review their purchases and track order details.

Functions:

- Place Order
 - Order Confirmation
 - View Order Details
 - Order History
-

6. Profile Management Module

This module allows customers to manage their personal information and account details.

Functions:

- View Profile
 - Update Profile Information
 - Manage Account Details
-

7. Admin Dashboard Module

The Admin Dashboard provides administrative control over the application. It enables administrators to manage products, categories, users, and orders efficiently.

Functions:

- Manage Products
 - Manage Categories
 - View Registered Users
 - Monitor Orders
 - Dashboard Overview
-

8. Database Module

The Database Module stores all application data using a JSON Database (db.json). It manages user information, products, categories, shopping cart details, and order records.

Functions:

- Store User Data
 - Store Product Information
 - Maintain Categories
 - Store Shopping Cart Data
 - Store Order Records
 - Perform CRUD Operations
-

Module Interaction

All modules work together to provide a complete online shopping experience. The User Authentication Module verifies users before granting access to the application. The Product and Category Management Modules allow customers to browse available products. Selected products are stored in the Shopping Cart Module, and once the customer confirms the purchase, the Order Management Module records the order details. The Profile Management Module enables users to maintain their account information, while the Admin Dashboard oversees the overall functioning of the application. All modules interact with the JSON Database through the backend to retrieve and update information efficiently.

Conclusion

The modular implementation of the Zivora – Online Shopping Platform improves code organization, simplifies maintenance, and supports future enhancements. Each module performs a specific responsibility while communicating seamlessly with other modules, resulting in a scalable, efficient, and user-friendly e-commerce application.

6.2– Front-End Logic

The front-end of the **Zivora – Online Shopping Platform** is developed using **React.js**, **Vite**, **HTML5**, **CSS3**, and **JavaScript (ES6)**. It provides an interactive, responsive, and user-friendly interface that enables customers and administrators to perform various shopping operations efficiently. The application follows a **component-based architecture**, where each page and feature is implemented as a reusable React component. This approach improves code maintainability, scalability, and overall application performance.

The frontend communicates with the backend using **RESTful APIs** to retrieve and update data stored in the **JSON Database (db.json)**. Navigation between different pages is handled using **React Router DOM**, allowing seamless page transitions without refreshing the browser.

1. Authentication Logic

The Authentication module handles user registration and login. It validates user credentials before allowing access to the application. Role-based authentication ensures that customers and administrators access only their respective functionalities.

Frontend Logic:

- Display login and registration forms.
 - Validate user input fields.
 - Send login or registration requests to the backend.
-

- Store authentication status after successful login.
 - Redirect users to the appropriate dashboard.
-

2. Product Display Logic

The Product module retrieves product information from the backend and displays it dynamically on the interface. Products are organized into categories for easier browsing.

Frontend Logic:

- Fetch product data using REST APIs.
 - Display product images, names, descriptions, and prices.
 - Organize products category-wise.
 - Update the product list dynamically whenever data changes.
-

3. Category Navigation Logic

This module allows users to browse products based on selected categories.

Frontend Logic:

- Display available product categories.
 - Filter products based on the selected category.
 - Refresh product listings dynamically without reloading the page.
-

4. Shopping Cart Logic

The Shopping Cart module enables users to manage products before placing an order.

Frontend Logic:

- Add selected products to the cart.
 - Increase or decrease product quantity.
 - Remove products from the cart.
 - Calculate and display the total amount.
 - Update cart information dynamically.
-

5. Order Processing Logic

This module manages the order confirmation process after the customer completes shopping.

Frontend Logic:

- Display selected cart items.
 - Calculate the final order amount.
 - Submit the order to the backend.
 - Display order confirmation.
 - Clear the shopping cart after successful order placement.
-

6. Profile Management Logic

The Profile module allows users to view and update their account information.

Frontend Logic:

- Display user profile information.
 - Validate updated details.
 - Submit profile changes to the backend.
 - Display confirmation after successful update.
-

7. Admin Dashboard Logic

The Admin Dashboard provides administrative functionalities for managing the application.

Frontend Logic:

- Display product management interface.
 - Display category management interface.
 - View customer information.
 - Monitor customer orders.
 - Update application data through backend APIs.
-

8. React State Management

The application uses **React Hooks** for managing dynamic data.

Hooks Used

- **useState()** – Manages component state such as login status, cart items, products, and user information.
 - **useEffect()** – Fetches data from the backend when components are loaded and updates the UI dynamically.
-

9. Routing Logic

The application uses **React Router DOM** for navigation between different pages.

The routing system connects pages such as:

- Home
- Login
- Register
- Products
- Categories
- Shopping Cart
- Orders
- Profile
- Admin Dashboard

This enables smooth navigation without refreshing the browser.

Advantages of the Front-End Logic

- Responsive and interactive user interface.
 - Component-based architecture.
 - Fast page rendering using React.js.
 - Smooth navigation using React Router DOM.
 - Dynamic content updates.
 - Easy maintenance and scalability.
 - Improved user experience.
 - Efficient communication with the backend.
-

Conclusion

The frontend logic of the **Zivora – Online Shopping Platform** is designed to provide a responsive, efficient, and user-friendly shopping experience. By utilizing **React.js**, **Vite**, **JavaScript**, **React Router DOM**, and **React Hooks**, the application ensures smooth navigation, dynamic content rendering, and efficient interaction with the backend. The modular implementation of frontend components improves maintainability and provides a strong foundation for future enhancements.

6.3 API Integration

API (Application Programming Interface) Integration is an essential part of the **Zivora – Online Shopping Platform**. It enables seamless communication between the **React.js frontend** and the **Node.js + Express.js backend**. The frontend sends HTTP requests to the backend through RESTful APIs, and the backend processes these requests, interacts with the **JSON Database (db.json)**, and returns the required data in **JSON format**.

The API integration allows different modules of the application, such as user authentication, product management, shopping cart, profile management, and order processing, to communicate efficiently. This separation of frontend and backend improves modularity, maintainability, and scalability of the application.

API Workflow

The API communication in the Zivora application follows these steps:

1. The user performs an action through the React.js interface.
2. The frontend sends an HTTP request to the Express.js server.
3. The backend validates the request.
4. The backend reads or updates data in the JSON Database (db.json).
5. The backend returns the response in JSON format.
6. The frontend receives the response and updates the user interface dynamically.

HTTP Methods Used

The application uses standard RESTful HTTP methods for data communication.

Method	Purpose
GET	Retrieve products, categories, users, and orders
POST	Register users, login, and place orders
PUT	Update user profile and shopping cart details
DELETE	Remove products from the shopping cart

API Modules

1. Authentication API

The Authentication API manages user registration and login.

Functions

- Register new users.
- Validate login credentials.
- Authenticate customers and administrators.
- Return user information after successful login.

2. Product API

The Product API retrieves product information stored in the database.

Functions

- Fetch all products.
- Display product details.
- Retrieve category-wise products.
- Support product search functionality.

3. Category API

The Category API provides product category information.

Functions

- Retrieve all categories.
 - Display products based on selected categories.
-

4. Shopping Cart API

The Shopping Cart API manages customer cart operations.

Functions

- Add products to cart.
 - Update product quantity.
 - Remove products from cart.
 - Retrieve shopping cart details.
-

5. Order API

The Order API processes customer orders.

Functions

- Place customer orders.
 - Retrieve order history.
 - Display order confirmation.
-

6. Profile API

The Profile API manages customer account information.

Functions

- Retrieve profile details.
 - Update customer information.
-

Benefits of API Integration

The API integration provides several advantages:

- Enables smooth communication between frontend and backend.
 - Supports dynamic data updates.
 - Improves application performance.
 - Simplifies maintenance.
 - Provides modular application architecture.
 - Supports future cloud deployment.
 - Easy integration with advanced databases and payment gateways.
-

Future API Enhancements

The application can be extended by integrating:

- Razorpay Payment API
 - Stripe Payment API
 - Firebase Authentication
-

- Cloudinary Image API
 - MongoDB Atlas
 - Email Notification API
-

Conclusion

The **API Integration** in the **Zivora – Online Shopping Platform** establishes effective communication between the React.js frontend and the Node.js + Express.js backend through RESTful APIs. These APIs manage user authentication, product retrieval, shopping cart operations, profile management, and order processing while interacting with the JSON Database. The modular API architecture improves scalability, maintainability, and performance, making the application easy to extend with advanced features such as online payments, cloud databases, and third-party services in future versions.

6.4 Authentication and Authorization

Authentication and Authorization are essential security mechanisms implemented in the **Zivora – Online Shopping Platform** to ensure that only authorized users can access the application and perform specific operations. Authentication verifies the identity of a user, while Authorization determines the resources and functionalities that the authenticated user is permitted to access. These mechanisms help maintain data security, prevent unauthorized access, and provide a secure shopping environment.

The application provides separate access for **Customers** and **Administrators**. Customers can browse products, manage their shopping cart, place orders, and update their profiles. Administrators have additional privileges such as managing products, categories, users, and customer orders through the Admin Dashboard.

6.4.1 Authentication

Authentication is the process of verifying the identity of a user before granting access to the system. The application provides secure registration and login functionality for both customers and administrators.

During registration, users provide their personal information such as name, email, password, phone number, and address. The system validates the entered information before creating a new account. During login, the entered credentials are verified against the user records stored in the **JSON Database (db.json)**. If the credentials are valid, the user is successfully authenticated and redirected to the appropriate page.

Authentication Process

1. User enters email and password.
 2. Frontend validates the input fields.
 3. Login request is sent to the backend through REST API.
 4. Backend verifies the credentials using the JSON Database.
 5. If the credentials are valid, the user is authenticated.
 6. The user is redirected to the Customer Dashboard or Admin Dashboard.
 7. Invalid credentials result in an error message requesting the user to try again.
-

6.4.2 Authorization

Authorization determines the actions that an authenticated user is allowed to perform. The application follows **Role-Based Access Control (RBAC)** by assigning different permissions to customers and administrators.

Customers have access only to shopping-related functionalities, while administrators have complete control over application management.

Customer Permissions

- Register and log in.
- Browse products.
- Search products.
- View product details.
- Add or remove items from the shopping cart.
- Place orders.
- View order history.
- Update personal profile.

Administrator Permissions

- Secure administrator login.
- Access Admin Dashboard.
- Manage products.
- Manage product categories.
- View registered users.
- View customer orders.
- Monitor application data.

Authentication and Authorization Workflow

The authentication and authorization process follows these steps:

1. The user accesses the login page.
2. The user enters valid credentials.
3. The backend authenticates the user.
4. The system identifies the user's role (Customer or Administrator).
5. Appropriate permissions are assigned.
6. The user gains access only to authorized modules.

7. Unauthorized access attempts are rejected.

Security Features

The application implements several security measures to improve system reliability:

- User authentication before accessing protected pages.
- Role-based authorization.
- Input validation on login and registration forms.
- Secure communication between frontend and backend using REST APIs.
- Prevention of unauthorized access to administrative modules.
- Session management after successful login.
- Logout functionality to terminate user sessions.

Advantages of Authentication and Authorization

- Protects user accounts from unauthorized access.
- Ensures secure login for customers and administrators.
- Restricts access based on user roles.
- Improves application security.
- Prevents unauthorized modification of application data.
- Provides better user management.
- Enhances overall system reliability.

Authentication and Authorization Summary

Feature	Implementation
Authentication	Email and Password Login
Authorization	Role-Based Access Control (RBAC)
User Roles	Customer, Administrator
Validation	Client-side and Server-side Validation
Data Storage	JSON Database (db.json)
Communication	REST APIs using Node.js and Express.js

Conclusion

Authentication and Authorization play a vital role in ensuring the security of the **Zivora – Online Shopping Platform**. Authentication verifies the identity of users before granting access, while authorization controls the functionalities available to each user based on their role. The implementation of **Role-Based Access Control (RBAC)**, secure login mechanisms, input validation, and protected administrative features ensures that the application remains secure, reliable, and user-friendly. This design also provides a strong foundation for integrating advanced security mechanisms such as **JWT authentication**, **OAuth**, and **multi-factor authentication (MFA)** in future versions of the application.

CHAPTER 7- FEATURES

7.1 List of Main Features

The Zivora – Online Shopping Platform provides a modern, user-friendly, and efficient solution for online shopping by integrating customer management, product browsing, shopping cart functionality, order processing, and administrative features into a single web-based application. The system enables customers to purchase products conveniently while allowing administrators to manage application data efficiently. The major features implemented in the system are described below.

1. User Authentication

The application provides secure customer registration and login functionality. Authentication ensures that only registered users can access their accounts, while administrators can securely log in to access the Admin Dashboard and manage the system.

2. Product Browsing

Customers can browse products from different categories such as Phones, Laptops, Watches, Bags, and Shoes. Each product displays its image, description, category, and price, allowing users to explore products before purchasing.

3. Category Management

Products are organized into different categories to improve navigation and simplify product searching. Users can browse category-wise products through an intuitive interface.

4. Product Search

The application provides a search feature that enables customers to quickly locate products by entering keywords. Matching products are displayed instantly, improving the shopping experience.

5. Shopping Cart

Customers can add products to their shopping cart, update quantities, remove unwanted items, and view the total purchase amount before placing an order.

6. Order Management

The application allows customers to place orders and maintain their order history. Order information is stored and displayed whenever customers access their accounts.

7. Profile Management

Customers can view and update their personal information, including profile details and account information, through the Profile section.

8. Admin Dashboard

The Admin Dashboard provides administrators with centralized control over the application. Administrators can monitor registered users, manage products and categories, and view customer orders.

9. Responsive User Interface

The application is developed using React.js and Vite, providing a responsive interface that works efficiently on desktops, laptops, tablets, and smartphones.

10. REST API Integration

The frontend communicates with the Node.js and Express.js backend through RESTful APIs, ensuring efficient data exchange and dynamic content updates.

Benefits of the Main Features

- Secure user registration and login.
- Easy browsing of multiple product categories.
- Fast product search functionality.
- Efficient shopping cart management.
- Simple and secure order placement.
- Responsive and user-friendly interface.
- Centralized administrator control.
- Easy maintenance through component-based architecture.
- Scalable design for future enhancements.

7.2 How Each Feature Works

This section explains the working of each feature from both the user's perspective and the technical implementation perspective.

1. User Authentication

User Perspective:

Customers register by entering their personal details and log in using their email and password. Administrators use a separate login to access the Admin Dashboard. Invalid login credentials display an appropriate error message.

Technical Description:

Authentication is implemented using React.js forms and React Router DOM for protected navigation. User credentials are validated through Node.js and Express.js APIs, which retrieve user information from the JSON Database (db.json). Based on the user's role, the system redirects them to either the customer interface or the Admin Dashboard.

2. Product Browsing

User Perspective:

Customers can browse products such as Phones, Laptops, Watches, Bags, and Shoes. Each product displays its image, description, price, and category.

Technical Description:

The Products.jsx component retrieves product information from the backend using REST APIs. React dynamically renders the product list using reusable components, allowing users to browse products efficiently.

3. Category Management

User Perspective:

Users can select a product category to view only the products belonging to that category.

Technical Description:

The Category component filters products based on the selected category. The frontend communicates with the backend through REST APIs to retrieve category-specific products and updates the interface dynamically.

4. Product Search

User Perspective:

Users enter a product name in the search bar. Matching products are displayed instantly, helping users locate items quickly.

Technical Description:

The search functionality filters products based on the entered keyword. React state management updates the displayed product list dynamically without refreshing the page.

5. Shopping Cart

User Perspective:

Customers add products to the shopping cart, update product quantities, remove unwanted items, and view the total purchase amount before checkout.

Technical Description:

The Cart component manages cart items using React state. Cart information is sent to the backend through REST APIs and stored in the JSON database. The total amount is calculated automatically whenever cart contents change.

6. Order Management

User Perspective:

After reviewing the shopping cart, customers place an order and receive an order confirmation. Previous orders can also be viewed from the account.

Technical Description:

The backend processes customer orders through the Orders API, stores order details in db.json, and returns confirmation information to the frontend for display.

7. Profile Management

User Perspective:

Customers can view and update their personal information such as name, phone number, address, and password.

Technical Description:

The Profile component retrieves user information from the backend and allows updates through REST API requests. Modified data is stored in the JSON database and reflected immediately in the user interface.

8. Admin Dashboard

User Perspective:

Administrators log in to access the dashboard where they can manage products, categories, registered users, and customer orders.

Technical Description:

The Admin Dashboard communicates with backend APIs to perform administrative operations. The dashboard retrieves and updates information stored in db.json, allowing administrators to manage the application efficiently.

Benefits of the Features

- Easy product browsing.
- Secure customer authentication.
- Efficient shopping cart management.
- Quick product searching.
- Simple order placement.
- Responsive user interface.
- Centralized administrator control.
- Fast API communication.
- Modular React architecture.
- Easy scalability for future enhancements.

Conclusion

The Zivora – Online Shopping Platform integrates customer authentication, product browsing, category management, shopping cart operations, order processing, profile management, and administrator functionalities into a single web-based application. By using React.js, Node.js, Express.js, and RESTful APIs, the application provides a responsive, secure, and efficient shopping experience. The modular implementation ensures that each feature works independently while seamlessly interacting with other modules, making the system easy to maintain and extend.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Software testing is one of the most important phases of the Software Development Life Cycle (SDLC). It is a systematic process of evaluating and verifying that a software application performs according to the specified requirements and functions correctly under different operating conditions. The primary objective of testing is to identify defects, improve software quality, ensure system reliability, and verify that the final application satisfies user requirements.

The Zivora – Online Shopping Platform was developed to provide customers with a simple and efficient online shopping experience. The application includes user registration, authentication, product browsing, category management, shopping cart operations, order processing, profile management, and administrative functionalities. Since the application contains multiple interconnected modules, it is essential to verify that each module functions correctly and communicates properly with the backend.

Testing was carried out throughout the development process to validate both the frontend and backend components of the application. The frontend, developed using React.js and Vite, was tested to ensure proper rendering of user interface components, navigation, responsive layouts, and form validations. The backend, developed using Node.js and Express.js, was tested using Postman to verify REST API endpoints, request handling, response generation, and interaction with the JSON Database (db.json).

Postman was used to send HTTP requests to various API endpoints and verify that the application returned the expected responses. The testing process included validating user authentication, product retrieval, category retrieval, shopping cart operations, order placement, and profile management APIs. Each endpoint was tested using appropriate HTTP methods such as GET, POST, PUT, and DELETE to ensure correct implementation of CRUD operations.

Both valid and invalid requests were executed to verify that the backend correctly handled successful operations as well as error conditions. Appropriate HTTP status codes and response messages were returned whenever invalid input or unauthorized access was detected. This testing helped identify and resolve issues before deployment and ensured smooth communication between the frontend and backend.

The application was also tested on different web browsers to verify consistent functionality and responsiveness across multiple platforms. All major modules performed successfully, confirming that the Zivora – Online Shopping Platform provides a reliable, secure, and user-friendly shopping experience.

Importance of Testing

Testing is important because it:

- Identifies software defects and implementation errors.
- Verifies frontend and backend communication.
- Validates REST API functionality.
- Ensures correct execution of CRUD operations.
- Improves software quality and reliability.
- Confirms that project requirements are satisfied.
- Enhances user experience and application performance.
- Reduces maintenance effort and development risks.
- Improves application security and stability.

- Ensures successful deployment of the system.
-

API Endpoints Tested

The following API modules were verified using Postman:

- User Registration API
 - User Login API
 - Product API
 - Category API
 - Shopping Cart API
 - Order API
 - Profile Management API
 - Admin Dashboard API
-

HTTP Methods Used During Testing

HTTP Method	Purpose
GET	Retrieve products, categories, cart items, orders, and user details
POST	Register users, authenticate login, add cart items, and place orders
PUT	Update profile information and modify shopping cart data
DELETE	Remove items from the shopping cart

Conclusion

The Postman Testing performed on the Zivora – Online Shopping Platform successfully verified the communication between the React.js frontend and the Node.js + Express.js backend. All major REST API endpoints responded correctly, CRUD operations were executed successfully, and the application behaved as expected under different test scenarios. The testing process confirmed that the system is reliable, secure, and ready for deployment while providing a smooth and efficient online shopping experience.

8.2 Integration Testing

Integration testing is performed to verify that all modules of the **Zivora – Online Shopping Platform** communicate correctly and work together as a single system. Unlike unit testing, which focuses on individual components, integration testing ensures that the interaction between the **React.js frontend**, **Node.js + Express.js backend**, and **JSON Database (db.json)** is accurate, reliable, and free from communication errors.

The primary objective of integration testing is to validate that data entered by the user flows correctly through different modules and that each module produces the expected output while interacting with other components. The testing process confirms that API requests, database operations, authentication, shopping cart management, order processing, and administrative functions work together seamlessly.

During integration testing, all major modules of the application were executed under normal operating conditions. The frontend communicated with the backend using **RESTful APIs**, while the backend processed requests and interacted with the JSON database to retrieve or update application data. The responses were then returned to the frontend and displayed dynamically without errors.

Functional Integration Testing

Functional integration testing was performed to verify the interaction between major application modules.

The following integrations were successfully tested:

- User Registration integrated with Login.
- Login integrated with Authentication.
- Authentication integrated with Profile Management.
- Product Module integrated with Category Module.
- Product Module integrated with Shopping Cart.
- Shopping Cart integrated with Order Management.
- Order Module integrated with User Profile.
- Admin Dashboard integrated with Product and Category Management.

The results confirmed that all integrated modules exchanged data correctly and performed the intended operations successfully.

User Interface Integration Testing

User Interface (UI) integration testing ensured that all frontend pages interacted correctly with backend services.

The following components were verified:

- Navigation Bar
- Login and Registration Forms
- Product Listing Page
- Category Navigation
- Shopping Cart Page
- Order Confirmation Page
- Profile Page
- Admin Dashboard

The testing confirmed smooth navigation between pages and correct display of dynamically retrieved data.

API Integration Testing

API integration testing verified communication between the frontend and backend.

The following API operations were tested:

- User Registration API
- Login API
- Product Retrieval API
- Category API
- Shopping Cart API
- Order API
- Profile Update API
- Admin Dashboard API

All APIs successfully exchanged JSON data and returned appropriate HTTP responses.

Database Integration Testing

Database integration testing verified that the backend correctly interacted with the **JSON Database (db.json)**.

The following operations were validated:

- User registration data storage.
- Product retrieval.
- Category retrieval.
- Shopping cart updates.
- Order storage.
- Profile updates.

All CRUD (Create, Read, Update, Delete) operations executed successfully without data inconsistencies.

Browser Compatibility Testing

The application was tested on multiple browsers to ensure consistent functionality.

Supported browsers include:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Testing confirmed that the user interface remained responsive and functioned correctly across all supported browsers.

Performance Testing

Basic performance testing was carried out under normal operating conditions.

The following parameters were evaluated:

- Page loading speed.
- Product retrieval speed.
- API response time.
- Shopping cart updates.
- Order processing time.
- User interface responsiveness.

The results demonstrated smooth application performance with acceptable response times.

Modules Tested

The following modules were integrated and verified successfully:

- Authentication Module
- Product Management Module
- Category Management Module
- Shopping Cart Module
- Order Management Module
- Profile Management Module
- Admin Dashboard Module
- REST API Module
- Database Module

Benefits of Integration Testing

Integration testing provided several advantages:

- Verified communication between frontend and backend.
- Ensured proper API functionality.
- Validated database operations.
- Improved application reliability.
- Reduced integration-related defects.
- Improved software quality.
- Enhanced user experience.
- Increased system stability.
- Confirmed successful interaction between all application modules.

Testing Environment

Component	Specification
Operating System	Windows 10 / Windows 11
Frontend	React.js, Vite
Backend	Node.js, Express.js
Database	JSON Database (db.json)
API Testing Tool	Postman
Browser	Google Chrome, Firefox, Microsoft Edge
Code Editor	Visual Studio Code

Conclusion

Integration testing confirmed that all modules of the **Zivora – Online Shopping Platform** communicate efficiently and operate as a unified system. The React.js frontend, Node.js + Express.js backend, and JSON database successfully exchanged data through RESTful APIs without errors. The successful execution of integration testing demonstrates that the application is reliable, stable, and capable of providing users with a seamless online shopping experience.

8.3 Tools Used for Testing

The Zivora – Online Shopping Platform was tested using various software tools and development environments to verify the functionality, performance, responsiveness, and reliability of the application. Different testing tools were used to validate the frontend, backend, APIs, database operations, and browser compatibility. These tools helped identify defects, debug issues, and ensure that the application functioned according to the specified requirements.

The following testing tools were used during the development of the project.

1. Postman

Postman was used to test the RESTful APIs developed using Node.js and Express.js. It allowed the testing of API endpoints independently of the frontend by sending HTTP requests and verifying responses.

Purpose

- Test REST APIs.
- Validate GET, POST, PUT, and DELETE requests.
- Verify HTTP status codes.
- Test JSON request and response data.
- Validate authentication APIs.

2. Google Chrome Developer Tools

Google Chrome Developer Tools were used to inspect the frontend, debug JavaScript code, monitor network requests, and analyze application performance.

Purpose

- Debug JavaScript.
- Inspect HTML and CSS.
- Monitor API requests.
- Detect frontend errors.
- Analyze page performance.

3. Visual Studio Code

Visual Studio Code was used as the primary development and debugging environment for both frontend and backend development.

Purpose

- Source code editing.
- Debugging.
- Error detection.
- Code formatting.
- Extension support.

4. React Developer Tools

React Developer Tools were used to inspect React components, monitor component hierarchy, and debug application state.

Purpose

- Inspect React components.
- Debug React Hooks.
- Analyze component state.
- Monitor props and state changes.

5. JSON Database (db.json)

The JSON Database was used to verify data storage and retrieval during testing.

Purpose

- Verify stored user information.
- Validate product records.
- Check shopping cart data.
- Verify order details.
- Test CRUD operations.

6. Web Browsers

Multiple browsers were used to verify browser compatibility and responsive behavior.

Browsers tested include:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Purpose

- Browser compatibility testing.
- Responsive UI testing.
- Navigation testing.
- Cross-browser validation.

Testing Environment

Component	Tool Used
Frontend Testing	React.js, Chrome DevTools
Backend Testing	Node.js, Express.js
API Testing	Postman
Database Testing	JSON Database (db.json)
Browser Testing	Chrome, Firefox, Edge
Code Editor	Visual Studio Code

Benefits of the Testing Tools

The testing tools provided several advantages during application development:

- Early detection of software defects.
- Efficient debugging of frontend and backend modules.
- Accurate API validation.
- Improved browser compatibility.
- Faster identification of coding errors.
- Verification of CRUD operations.
- Enhanced software quality and reliability.
- Improved application performance and stability.

Conclusion

The testing tools used during the development of the Zivora – Online Shopping Platform played a significant role in ensuring software quality, functionality, and reliability. Postman verified REST API communication, Chrome Developer Tools and React Developer Tools assisted in debugging the

frontend, Visual Studio Code supported efficient development, and browser compatibility testing ensured consistent performance across multiple browsers. Together, these tools helped deliver a stable, responsive, and user-friendly online shopping application.

8.4 Test Cases and Results

The testing phase of the **Zivora – Online Shopping** Platform was conducted systematically to verify that every module of the application performed according to the specified requirements. Test cases were prepared for all major functionalities, including user authentication, product browsing, category management, shopping cart operations, order processing, profile management, and administrative features.

Each test case was executed under normal operating conditions, and the obtained results were compared with the expected outcomes. The successful execution of these test cases confirmed that the application functions correctly and provides a reliable online shopping experience.

Test Case Results

Test

Case ID	Module	Test Scenario	Expected Result	Actual Result	Status
TC-01	User Registration	Register with valid user details	User account should be created successfully	User account created successfully	Pass
TC-02	User Login	Login with valid credentials	User should be redirected to the Home page	Login successful	Pass
TC-03	User Login	Login with invalid credentials	Error message should be displayed	Error message displayed	Pass
TC-04	Product Module	View all products	Product list should be displayed	Products displayed correctly	Pass
TC-05	Category Module	Select a product	Category-wise products should	Products filtered	Pass

Test Case ID	Module	Test Scenario	Expected Result	Actual Result	Status
		category	be displayed	correctly	
TC-06	Product Search	Search for a product	Matching products should appear	Search results displayed correctly	Pass
TC-07	Shopping Cart	Add a product to the cart	Product should be added successfully	Product added successfully	Pass
TC-08	Shopping Cart	Remove a product from the cart	Product should be removed successfully	Product removed successfully	Pass
TC-09	Shopping Cart	Update product quantity	Cart total should update automatically	Quantity updated correctly	Pass
TC-10	Order Management	Place an order	Order should be stored successfully	Order placed successfully	Pass
TC-11	Profile Management	Update profile information	Profile should be updated	Profile updated successfully	Pass
TC-12	Admin Dashboard	Admin login and dashboard access	Dashboard should open successfully	Dashboard displayed successfully	Pass

Testing Summary

Parameter	Result
Total Test Cases Executed	12
Test Cases Passed	12

Parameter	Result
Test Cases Failed	0
Success Rate	100%

Observations

The testing results indicate that all major modules of the Zivora – Online Shopping Platform operated successfully under normal operating conditions. The frontend and backend communicated efficiently through RESTful APIs, and all CRUD operations were executed correctly using the JSON database. User authentication, product browsing, shopping cart management, order processing, profile updates, and administrative operations performed as expected without critical errors.

Conclusion

The successful execution of all test cases confirms that the Zivora – Online Shopping Platform satisfies its functional requirements and provides a stable, reliable, and user-friendly shopping experience. The testing process validated the correctness of the application's frontend, backend, REST APIs, and database interactions. The results demonstrate that the system is ready for deployment and can be further enhanced with additional features such as online payment integration, cloud deployment, and advanced security mechanisms in future versions.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

Deployment is the process of making the application available for end users after development and testing have been completed. The Zivora – Online Shopping Platform follows a client-server architecture where the frontend, backend, and database are configured to work together. During deployment, all project dependencies are installed, the backend server is started, and the frontend application is launched to provide users with access to the shopping platform.

The application is developed using React.js and Vite for the frontend, Node.js and Express.js for the backend, and a JSON Database (db.json) for data storage. The deployment process ensures that all components communicate correctly through RESTful APIs and provide a smooth user experience.

Deployment Steps

Step 1: Install Prerequisites

Install the required software before running the project.

- Node.js
- npm (Node Package Manager)
- Visual Studio Code
- Git (optional)

Step 2: Download the Project

Clone the project from GitHub or download the project source code.

```
git clone <repository-url>
```

or

Download the ZIP file and extract it.

Step 3: Install Dependencies

Open the project folder in Visual Studio Code and install all required packages.

```
npm install
```

This command installs all dependencies listed in the package.json file.

Step 4: Start the Backend Server

Navigate to the backend directory and start the Express server.

```
npm start
```

or

```
node server.js
```

The backend server starts and waits for API requests from the frontend.

Step 5: Start the Frontend

Navigate to the frontend project folder and run:

`npm run dev`

The Vite development server starts and provides a local URL such as:

`http://localhost:5173`

Step 6: Access the Application

Open the provided URL in a web browser.

The application loads successfully and users can:

- Register
 - Login
 - Browse products
 - Add products to cart
 - Place orders
 - Manage profile
 - Access the Admin Dashboard (Administrator only)
-

Step 7: Verify Application

Verify that all modules are working correctly.

Check:

- User Registration
 - User Login
 - Product Listing
 - Product Categories
 - Shopping Cart
 - Order Placement
 - Profile Management
 - Admin Dashboard
 - API Communication
-

Deployment Workflow

Project Source Code

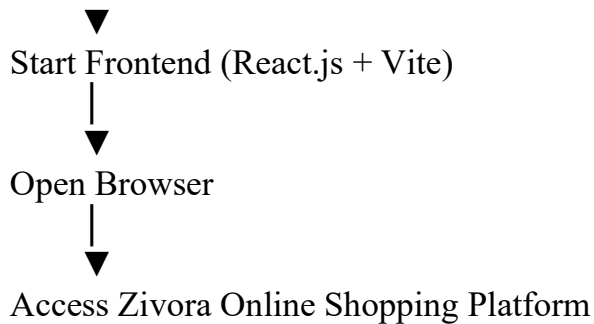


Install Dependencies (`npm install`)



Start Backend (`Node.js + Express.js`)





Advantages of the Deployment Process

- Simple deployment procedure.
- Easy dependency management using npm.
- Fast application startup using Vite.
- Efficient backend communication through Express.js.
- Easy debugging during development.
- Modular project structure.
- Supports future cloud deployment.

Conclusion

The deployment process of the Zivora – Online Shopping Platform is simple and efficient. By installing the required dependencies, starting the backend server, and launching the React.js frontend, the application becomes fully functional and ready for use. The deployment approach ensures smooth communication between the frontend, backend, and JSON database, providing users with a reliable and responsive online shopping experience. The same architecture can be extended for deployment on cloud platforms such as Vercel, Render, or AWS in future versions.

9.2 Environment Configuration

Environment configuration is an essential step in deploying and executing the **Zivora – Online Shopping Platform**. It involves setting up the necessary software, development tools, libraries, and project dependencies required for the frontend, backend, and database to work together efficiently. A properly configured environment ensures smooth development, testing, and deployment of the application.

The project is developed using **React.js** and **Vite** for the frontend, **Node.js** and **Express.js** for the backend, and a **JSON Database (db.json)** for storing application data. All required packages are installed using **Node Package Manager (npm)**.

Frontend Environment

The frontend environment is responsible for rendering the user interface and handling user interactions.

Configuration:

- Framework: React.js
- Build Tool: Vite

- Programming Language: JavaScript (ES6)
- Markup Language: HTML5
- Styling: CSS3
- Routing: React Router DOM
- Package Manager: npm

Frontend Setup:

```
npm install  
npm run dev
```

The frontend application runs locally using the Vite development server.

Backend Environment

The backend handles business logic, authentication, API requests, and communication with the database.

Configuration:

- Runtime Environment: Node.js
- Backend Framework: Express.js
- API Type: RESTful APIs
- Package Manager: npm

Backend Setup:

```
npm install  
npm start
```

The backend server processes requests received from the frontend and returns responses in JSON format.

Database Configuration

The project uses a lightweight **JSON Database (db.json)** to store application data.

The database stores:

- User Information
- Product Details
- Product Categories
- Shopping Cart Data
- Order Information

The backend performs CRUD (Create, Read, Update, Delete) operations on the JSON database whenever required.

Development Environment

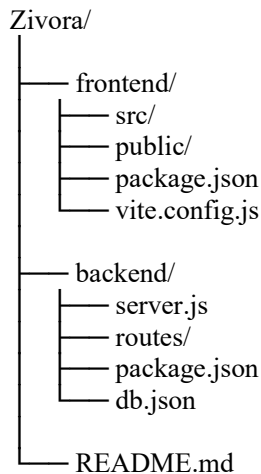
The following development tools were used during implementation.

Software	Version
Operating System	Windows 10 / Windows 11
Code Editor	Visual Studio Code
Frontend Framework	React.js
Build Tool	Vite
Backend Framework	Express.js
Runtime Environment	Node.js
Database	JSON Database (db.json)
Package Manager	npm

Software	Version
API Testing Tool	Postman
Version Control	Git & GitHub
Browser	Google Chrome / Firefox / Microsoft Edge

Project Structure

The project is organized into separate frontend and backend modules.



Advantages of Environment Configuration

- Easy project setup.
- Organized project structure.
- Simple dependency management using npm.
- Efficient frontend and backend communication.
- Easy debugging and maintenance.
- Supports future upgrades and cloud deployment.
- Improves development productivity.

Conclusion

A properly configured environment ensures that the **Zivora – Online Shopping Platform** operates efficiently during development and deployment. By using **React.js**, **Vite**, **Node.js**, **Express.js**, and a **JSON Database**, the application provides a stable and scalable architecture. The modular environment setup simplifies maintenance, supports efficient debugging, and enables future enhancements such as cloud deployment and advanced database integration.

9.3 Hosting of Frontend and Backend

Hosting is the process of deploying an application on a server so that it can be accessed by users over the internet. The **Zivora – Online Shopping Platform** follows a client-server architecture in which the frontend and backend can be hosted independently. This approach improves scalability, maintainability, and overall application performance.

The frontend of the application is developed using **React.js** and **Vite**, while the backend is developed using **Node.js** and **Express.js**. During development, both components are executed locally. For production deployment, the frontend and backend can be hosted on cloud platforms, enabling users to access the application from anywhere with an internet connection.

Frontend Hosting

The frontend is responsible for displaying the user interface and handling user interactions. Since it is developed using **React.js** and **Vite**, it can be deployed to static hosting platforms.

Recommended Hosting Platforms

- Vercel
- Netlify
- GitHub Pages

Frontend Deployment Process

1. Build the React application using:
`npm run build`
 2. Generate the production build.
 3. Upload the generated build files to the hosting platform.
 4. Configure the deployment settings.
 5. Access the application using the generated deployment URL.
-

Backend Hosting

The backend processes business logic, handles API requests, and communicates with the database.

The backend is developed using:

- Node.js
- Express.js

Recommended Hosting Platforms

- Render
- Railway
- AWS EC2
- Microsoft Azure
- Google Cloud Platform (GCP)

Backend Deployment Process

1. Upload the backend source code.
 2. Install project dependencies.
-

3. Configure environment variables if required.
4. Start the Node.js server.
5. Verify that REST APIs are functioning correctly.

Database Hosting

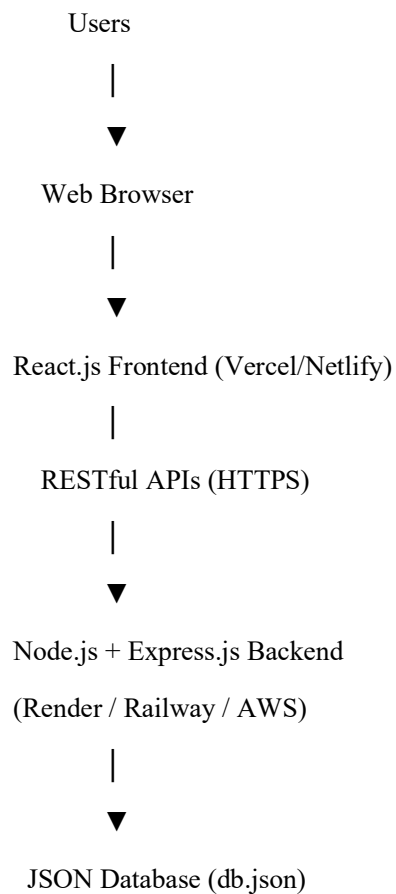
The current version of the project uses a **JSON Database (db.json)** for storing application data. Since the application is developed for academic purposes, the database is maintained locally with the backend.

For future deployment, the database can be migrated to cloud-based database services such as:

- MongoDB Atlas
- MySQL
- PostgreSQL

These databases provide improved scalability, security, and data management capabilities.

Hosting Architecture



Advantages of Cloud Hosting

- High application availability.
 - Improved scalability.
 - Faster application performance.
 - Independent frontend and backend deployment.
 - Easy maintenance and updates.
 - Secure access through HTTPS.
 - Better reliability and fault tolerance.
 - Supports future database migration.
-

Future Hosting Enhancements

The hosting architecture can be further improved by:

- Deploying the frontend on **Vercel**.
 - Hosting the backend on **Render** or **AWS EC2**.
 - Migrating from **db.json** to **MongoDB Atlas**.
 - Configuring automated deployment using **GitHub Actions**.
 - Enabling HTTPS with SSL certificates.
 - Implementing automatic backups and monitoring.
-

Conclusion

The **Zivora – Online Shopping Platform** supports independent hosting of its frontend and backend components, making the application scalable and easy to maintain. The React.js frontend can be hosted on platforms such as **Vercel** or **Netlify**, while the Node.js and Express.js backend can be deployed on **Render**, **Railway**, or **AWS**. Although the current version uses a local **JSON Database (db.json)**, the architecture is designed to support migration to cloud-based databases in future versions. This deployment approach ensures a reliable, secure, and efficient environment for delivering the online shopping application.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

During the development of the **Zivora – Online Shopping Platform**, several technical and implementation challenges were encountered. These challenges were addressed through debugging, testing, code optimization, and continuous improvement of the application. Overcoming these issues helped improve the stability, functionality, and overall quality of the project.

The major challenges faced during development are discussed below.

1. User Authentication

Implementing secure user registration and login required proper validation of user credentials and role-based access. Ensuring that customers and administrators accessed only their respective modules was an important challenge.

2. Frontend–Backend Communication

Establishing seamless communication between the **React.js** frontend and the **Node.js + Express.js** backend required correct API endpoint configuration and proper handling of HTTP requests and responses.

3. Shopping Cart Management

Managing shopping cart operations such as adding products, updating quantities, removing items, and calculating the total amount dynamically required careful state management and synchronization with the backend.

4. Dynamic Data Rendering

Displaying product information, categories, shopping cart contents, and user data dynamically without refreshing the page required effective use of React components and state management.

5. API Debugging

Several API requests initially returned incorrect responses due to routing errors, invalid request formats, and improper data handling. These issues were resolved through extensive testing using **Postman**.

6. Database Management

Maintaining consistent data in the **JSON Database (db.json)** while performing CRUD operations required careful validation to prevent duplicate records and incorrect updates.

7. Responsive User Interface

Designing a responsive interface that functioned correctly across desktops, laptops, tablets, and mobile devices required multiple layout adjustments and browser compatibility testing.

8. State Management

Managing application state across multiple React components, particularly for shopping cart updates, user sessions, and product listings, required careful implementation using React Hooks such as **useState** and **useEffect**.

9. Browser Compatibility

The application was tested on **Google Chrome**, **Mozilla Firefox**, and **Microsoft Edge** to ensure consistent behavior. Minor styling differences and rendering issues were identified and corrected.

10. Debugging and Error Handling

Identifying runtime errors, fixing JavaScript exceptions, handling API failures, and displaying meaningful error messages were important parts of the development process.

Challenges Summary

Challenge	Impact
User Authentication	Secure access control
Frontend–Backend Communication	Reliable API interaction
Shopping Cart Management	Accurate cart operations
Dynamic Data Rendering	Smooth user experience
API Debugging	Correct API functionality
Database Management	Reliable data storage
Responsive Design	Cross-device compatibility
State Management	Efficient UI updates
Browser Compatibility	Consistent application behavior
Error Handling	Improved software reliability

Conclusion

The development of the **Zivora – Online Shopping Platform** involved several technical challenges related to authentication, frontend-backend integration, API communication, state management, responsive design, and database operations. By systematically identifying and resolving these issues through debugging, testing, and optimization, the project was successfully completed. Overcoming these challenges not only improved the quality and reliability of the application but also provided valuable practical experience in full-stack web development.

10.2 Solutions Applied

During the development of the **Zivora – Online Shopping Platform**, various technical challenges were encountered while implementing user authentication, frontend-backend communication, shopping cart management, dynamic data rendering, and API integration. Appropriate solutions were applied to resolve these challenges and ensure that the application functioned efficiently, securely, and reliably. These solutions improved system performance, enhanced user experience, and increased the maintainability of the application.

The major solutions implemented during development are described below.

1. Secure User Authentication

To ensure secure access to the application, user registration and login functionality were implemented with proper input validation. Separate roles were maintained for **Customers** and **Administrators**, ensuring that users could only access the features authorized for their role.

2. REST API Integration

Communication between the **React.js** frontend and the **Node.js + Express.js** backend was established using **RESTful APIs**. Standard HTTP methods (**GET**, **POST**, **PUT**, and **DELETE**) were used to perform CRUD operations efficiently.

3. Modular Component Design

The frontend was developed using reusable **React components**, making the application easier to maintain, debug, and extend. Each module was designed independently while ensuring seamless interaction with other modules.

4. State Management Using React Hooks

React Hooks such as **useState()** and **useEffect()** were used to manage dynamic data including user information, product listings, shopping cart contents, and order details. This enabled real-time updates without refreshing the page.

5. Database Management

The **JSON Database (db.json)** was used to store users, products, categories, shopping cart information, and order details. CRUD operations were implemented through the backend to ensure reliable data storage and retrieval.

6. Shopping Cart Optimization

The shopping cart was designed to automatically update product quantities and calculate the total purchase amount whenever items were added, removed, or modified. This improved the overall shopping experience.

7. Responsive User Interface

Responsive web design techniques using **HTML5**, **CSS3**, and **React.js** ensured that the application worked correctly on desktops, laptops, tablets, and mobile devices. Browser compatibility testing further improved the user interface.

8. API Testing and Debugging

Postman was used to verify all backend API endpoints. Incorrect responses, routing errors, and request handling issues were identified and resolved before deployment, ensuring reliable communication between the frontend and backend.

9. Error Handling

Proper validation and error handling mechanisms were implemented throughout the application. Invalid inputs, login failures, and unsuccessful API requests display meaningful error messages to guide users and improve application reliability.

Solutions Summary

Challenge	Solution Applied
User Authentication	Implemented secure login and role-based access
Frontend–Backend Communication	Integrated RESTful APIs
Shopping Cart Management	Dynamic cart updates and total calculation
Dynamic Data Rendering	React Components with Hooks
Database Management	CRUD operations using JSON Database
Responsive Design	Responsive layout using HTML5, CSS3, and React.js
API Errors	Debugged using Postman
Error Handling	Implemented input validation and exception handling

Benefits of the Applied Solutions

The implemented solutions provided several advantages:

- Improved application security.
 - Efficient frontend-backend communication.
 - Better code organization through modular components.
 - Dynamic user interface updates.
 - Reliable database operations.
 - Enhanced user experience.
 - Improved application stability.
 - Simplified maintenance and future enhancements.
-

Conclusion

The solutions implemented during the development of the **Zivora – Online Shopping Platform** successfully addressed the challenges encountered throughout the project. By adopting a modular architecture, integrating RESTful APIs, implementing React Hooks for state management, and using proper validation and testing techniques, the application achieved reliable performance, efficient communication, and a responsive user interface. These solutions established a strong foundation for future enhancements such as cloud deployment, online payment integration, and advanced database management.

10.3 Current Limitations or Known Bugs

Although the **Zivora – Online Shopping Platform** successfully implements the core functionalities of an online shopping application, there are certain limitations in the current version. These limitations are primarily due to the project's academic scope, limited development time, and the use of lightweight technologies such as a **JSON Database (db.json)**. These areas provide opportunities for future improvements and enhancements.

The current limitations of the system are discussed below.

1. JSON Database Limitation

The application uses a **JSON Database (db.json)** for data storage. While suitable for development and academic purposes, it is not ideal for handling large amounts of data, concurrent users, or enterprise-level applications.

2. No Online Payment Integration

The current version does not include an online payment gateway. Customers can place orders, but secure online payment services such as **Razorpay**, **Stripe**, or **PayPal** are not integrated.

3. Limited Product Search

The search functionality supports basic keyword searching. Advanced search features such as filtering by price, ratings, popularity, or brand are not available.

4. No Product Review and Rating System

Customers cannot submit product reviews or ratings. Therefore, buyers cannot view feedback from other customers before making purchasing decisions.

5. Basic Security Implementation

The application provides user authentication and role-based authorization. However, advanced security mechanisms such as **JWT authentication**, **OAuth**, **Two-Factor Authentication (2FA)**, and encrypted password storage are not implemented in the current version.

6. Local Database Storage

Since the application stores data in **db.json**, information is maintained locally. The current version does not use cloud-hosted databases such as **MongoDB Atlas**, **MySQL**, or **PostgreSQL**, limiting scalability and data availability.

7. No Real-Time Notifications

Users do not receive real-time notifications for order status updates, promotional offers, or administrative announcements.

8. Limited Admin Analytics

The Admin Dashboard provides basic management functionality but does not include advanced analytics, sales reports, inventory statistics, or graphical dashboards.

9. Limited Deployment

The application is primarily developed and tested in a local environment. Full cloud deployment, automatic scaling, and production monitoring have not yet been implemented.

10. Scalability Constraints

The current architecture is suitable for small-scale applications. Additional optimization and enterprise-grade infrastructure would be required to support thousands of concurrent users.

Summary of Current Limitations

Limitation	Description
JSON Database	Suitable only for small-scale applications
Online Payment	Payment gateway not integrated
Product Search	Basic keyword search only
Reviews & Ratings	Not implemented
Advanced Security	JWT, OAuth, and 2FA not implemented
Cloud Database	Uses local JSON database
Notifications	No real-time notifications
Admin Analytics	Basic dashboard only
Cloud Deployment	Local deployment only
Scalability	Limited support for high user traffic

Future Scope to Overcome These Limitations

The identified limitations can be addressed in future versions by:

- Migrating the database to **MongoDB Atlas** or **MySQL**.
 - Integrating **Razorpay** or **Stripe** for secure online payments.
 - Implementing advanced product search and filtering.
 - Adding customer reviews and product ratings.
 - Implementing **JWT authentication** and stronger security measures.
 - Enabling real-time notifications using **WebSockets** or **Firebase**.
 - Developing advanced analytics and reporting features for administrators.
 - Deploying the application on cloud platforms such as **Vercel**, **Render**, or **AWS**.
 - Improving scalability through optimized backend architecture and cloud databases.
-

Conclusion

The **Zivora – Online Shopping Platform** successfully fulfills its primary objective of providing an efficient and user-friendly online shopping experience. However, the current version has limitations related to payment integration, advanced security, scalability, cloud deployment, and analytics. These limitations do not affect the core functionality of the application but represent opportunities for future enhancement. With the integration of advanced technologies and cloud-based services, the platform can be transformed into a robust, scalable, and production-ready e-commerce solution suitable for real-world deployment.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The current version of the **Zivora – Online Shopping Platform** successfully provides the essential functionalities required for an online shopping application, including user authentication, product browsing, shopping cart management, order processing, profile management, and administrative operations. However, there are several opportunities to enhance the system by integrating advanced technologies and additional features that improve usability, security, scalability, and overall user experience. The following features are planned for future versions of the application.

1. Online Payment Gateway Integration

Future versions of the application will support secure online payment gateways such as **Razorpay**, **Stripe**, or **PayPal**. This will allow customers to complete purchases using credit cards, debit cards, UPI, and digital wallets, making the shopping experience more convenient and secure.

2. Product Reviews and Ratings

A review and rating system will be introduced to enable customers to share their feedback on purchased products. This feature will help new customers make informed purchasing decisions and improve overall product quality.

3. Advanced Product Search and Filtering

The application will include advanced search and filtering options based on product category, price range, brand, popularity, customer ratings, and availability. These enhancements will help users quickly locate products that match their preferences.

4. Wishlist Functionality

Customers will be able to save products in a personal wishlist for future purchases. This feature will improve user engagement and provide a better shopping experience.

5. Order Tracking System

A real-time order tracking system will allow customers to monitor the status of their orders, including order confirmation, processing, shipping, and delivery.

6. Notification System

The application will provide real-time notifications for order updates, promotional offers, discounts, and important announcements using email or push notifications.

7. Cloud Database Integration

The existing **JSON Database (db.json)** will be replaced with a cloud-based database such as **MongoDB Atlas**, **MySQL**, or **PostgreSQL** to improve scalability, performance, and data security.

8. Enhanced Security Features

Future versions will include advanced security mechanisms such as **JWT authentication**, **password encryption**, **Two-Factor Authentication (2FA)**, and **OAuth-based login** to improve user account protection.

9. Sales Analytics Dashboard

The Admin Dashboard will be enhanced with graphical reports and analytics, allowing administrators to monitor sales, customer activity, product performance, and inventory levels through charts and reports.

10. Mobile Application

A dedicated Android and iOS mobile application will be developed using **React Native** or **Flutter**, allowing customers to access the shopping platform from mobile devices with the same features available in the web application.

Summary of Planned Features

Planned Feature	Expected Benefit
Online Payment Gateway	Secure online transactions
Product Reviews & Ratings	Better customer feedback
Advanced Search & Filtering	Faster product discovery
Wishlist	Improved shopping experience
Order Tracking	Real-time order updates
Notification System	Instant user alerts
Cloud Database	Better scalability and security
Enhanced Security	Stronger user protection
Sales Analytics Dashboard	Improved business insights
Mobile Application	Access from Android and iOS devices

Benefits of Planned Features

The implementation of these planned features will provide several advantages:

- Improved user experience.
- Secure online payment processing.
- Better customer engagement.
- Faster product discovery.
- Enhanced application security.
- Improved scalability and performance.
- Better administrative control.
- Increased customer satisfaction.
- Support for mobile users.
- Readiness for real-world deployment.

Conclusion

The planned features outlined above will significantly enhance the capabilities of the **Zivora – Online Shopping Platform**. By integrating secure payment systems, advanced search, customer reviews, cloud databases, enhanced security mechanisms, and mobile application support, the platform can evolve into a comprehensive and scalable e-commerce solution. These enhancements will improve usability, reliability, and overall customer satisfaction while preparing the application for large-scale commercial deployment.

11.2 Possible Integrations or Optimizations

The **Zivora – Online Shopping Platform** has been successfully developed as a web-based e-commerce application that provides essential functionalities such as user authentication, product browsing, shopping cart management, order processing, profile management, and administrative control. Although the current implementation fulfills the primary objectives of the project, several enhancements can be incorporated in future versions to improve performance, security, scalability, and user experience.

One of the primary recommendations is to migrate the application from the current **JSON Database (db.json)** to a more robust database management system such as **MongoDB**, **MySQL**, or **PostgreSQL**. This will enable efficient handling of large volumes of data, improve performance, and support multiple concurrent users.

Another important recommendation is to integrate a secure **online payment gateway** such as **Razorpay**, **Stripe**, or **PayPal**. This will allow customers to complete transactions securely using debit cards, credit cards, UPI, and digital wallets, making the application suitable for real-world commercial use.

The security of the application can be further strengthened by implementing **JWT (JSON Web Token) authentication**, password encryption, **OAuth-based login**, and **Two-Factor Authentication (2FA)**. These enhancements will improve user account protection and reduce the risk of unauthorized access.

Future versions of the application should also include **customer reviews and product ratings**, enabling users to share their experiences and helping other customers make informed purchasing decisions. In addition, implementing **advanced search and filtering** based on price, brand, ratings, and availability will improve product discovery and overall user satisfaction.

The application can also benefit from a **real-time notification system** that informs customers about order confirmations, shipping updates, promotional offers, and important announcements through email or push notifications.

To improve business management, the **Admin Dashboard** can be enhanced by adding sales reports, inventory management, graphical analytics, and customer behavior analysis. These features will assist administrators in making informed business decisions.

Another recommendation is to develop a dedicated **mobile application** using **React Native** or **Flutter**, allowing customers to access the platform conveniently from Android and iOS devices. This will improve accessibility and increase user engagement.

Finally, deploying the application on cloud platforms such as **Vercel** (frontend) and **Render** or **AWS** (backend) will improve scalability, availability, and performance. Automated deployment using **CI/CD** pipelines and cloud databases will make the system suitable for enterprise-level applications.

Recommendations Summary

Recommendation	Expected Benefit
Migrate to MongoDB/MySQL/PostgreSQL	Better scalability and performance
Integrate Online Payment Gateway	Secure online transactions
Implement JWT & 2FA	Improved application security
Add Reviews & Ratings	Better customer feedback
Advanced Search & Filters	Improved product discovery
Real-Time Notifications	Better user engagement
Enhance Admin Dashboard	Improved business analytics
Develop Mobile Application	Better accessibility
Cloud Deployment	High availability and scalability
CI/CD Implementation	Automated deployment and maintenance

Conclusion

The **Zivora – Online Shopping Platform** provides a strong foundation for an e-commerce system, but there are several opportunities for future improvements. By implementing the recommended enhancements, including cloud-based databases, secure payment integration, advanced security mechanisms, mobile application support, and improved analytics, the application can evolve into a scalable, secure, and enterprise-ready e-commerce solution capable of meeting real-world business requirements.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The **Zivora – Online Shopping Platform** was successfully designed and developed as a modern web-based e-commerce application that provides a simple, secure, and user-friendly shopping experience. The project achieved its primary objective of creating an integrated platform where customers can browse products, manage shopping carts, place orders, and maintain their profiles, while administrators can efficiently manage products, categories, users, and customer orders through a dedicated Admin Dashboard.

The application was implemented using **React.js**, **Vite**, **Node.js**, **Express.js**, and a **JSON Database (db.json)**. The frontend provides an interactive and responsive user interface, while the backend handles business logic, RESTful API communication, and database operations. The modular architecture adopted in the project improves code organization, maintainability, and scalability, making the application suitable for future enhancements.

Throughout the development process, all major modules, including **User Authentication**, **Product Management**, **Category Management**, **Shopping Cart**, **Order Management**, **Profile Management**, and **Admin Dashboard**, were successfully implemented and tested. Integration testing and API testing confirmed that the system functions correctly and provides reliable communication between the frontend, backend, and database.

Although the current version uses a lightweight **JSON Database** and is intended primarily for academic purposes, the project establishes a strong foundation for developing a full-scale e-commerce platform. Future enhancements such as cloud database integration, secure online payment gateways, advanced security mechanisms, customer reviews, mobile application support, and real-time notifications can further improve the functionality and scalability of the system.

In conclusion, the **Zivora – Online Shopping Platform** successfully demonstrates the practical application of modern full-stack web development technologies and fulfills the objectives defined at the beginning of the project. The project has enhanced understanding of frontend development, backend programming, RESTful APIs, database management, and software engineering practices while providing a scalable framework for future development.

12.2 What Was Achieved

The **Zivora – Online Shopping Platform** successfully achieved the objectives defined during the project planning and development phases. The project demonstrates the implementation of a modern full-stack web application by integrating **React.js**, **Vite**, **Node.js**, **Express.js**, and a **JSON Database (db.json)** into a single, functional e-commerce system.

The application provides a secure and user-friendly environment where customers can register, log in, browse products, search by category, manage their shopping cart, place orders, and update their profiles. Administrators are provided with a dedicated dashboard to manage products, categories, users, and customer orders efficiently.

The project successfully implemented a responsive frontend using **React.js**, allowing users to navigate smoothly between pages without reloading the application. The backend was developed using **Node.js** and **Express.js**, enabling efficient processing of business logic and communication with the database through **RESTful APIs**. The JSON database was successfully used to store and retrieve application data, including users, products, categories, shopping cart information, and orders.

The system was tested thoroughly using **Postman** and browser-based testing to verify the correctness of API communication, user authentication, product management, shopping cart functionality, order processing, and profile management. The successful execution of all test cases confirmed that the application performs according to the specified requirements.

In addition to meeting the functional requirements, the project enhanced practical knowledge of frontend development, backend programming, REST API development, database management, software testing, debugging, and deployment techniques. The modular architecture adopted in the project improves maintainability, scalability, and future extensibility.

Although the current version is intended for academic purposes, the project establishes a strong foundation for future enhancements such as cloud database integration, secure online payment gateways, advanced security mechanisms, real-time notifications, customer reviews, and mobile application support.

Achievements Summary

The major achievements of the project include:

- Successfully developed a complete online shopping web application.
- Implemented secure user registration and login functionality.
- Developed a responsive user interface using **React.js** and **Vite**.
- Implemented backend services using **Node.js** and **Express.js**.
- Integrated frontend and backend using **RESTful APIs**.
- Implemented product and category management.
- Developed shopping cart and order management modules.
- Implemented profile management for customers.
- Developed an Admin Dashboard for system administration.
- Successfully stored and managed application data using **JSON Database (db.json)**.
- Performed API testing using **Postman**.

- Verified application functionality through integration and system testing.
 - Designed the application using a modular and scalable architecture.
 - Established a strong foundation for future enhancements and cloud deployment.
-

Conclusion

The **Zivora – Online Shopping Platform** successfully fulfills its intended objectives by providing a functional, responsive, and secure e-commerce application. The project demonstrates the practical implementation of modern web technologies and software engineering principles while addressing the essential requirements of an online shopping system. The successful completion of this project reflects the effective application of full-stack development concepts and provides a scalable foundation for future improvements and real-world deployment.

12.3 Skills Learned During Development

The development of the **Zivora – Online Shopping Platform** provided valuable practical experience in modern web application development. Throughout the project, various technical and professional skills were acquired while designing, implementing, testing, and documenting the system. The project enhanced understanding of full-stack web development, software engineering principles, project management, and collaborative problem-solving.

Full-Stack Web Development Skills

The project provided hands-on experience in developing a complete web application using modern frontend and backend technologies.

Skills Learned

- React.js component-based development
- JavaScript (ES6) programming
- HTML5 page structure
- CSS3 responsive web design
- Node.js backend development
- Express.js REST API development
- JSON database management
- Frontend and backend integration

Benefits

- Improved understanding of modern web development.
 - Ability to develop complete full-stack applications.
 - Better knowledge of component-based architecture.
 - Enhanced code reusability and maintainability.
-

Frontend Development Skills

The frontend implementation helped improve user interface development and responsive web design.

Skills Learned

- React Components
- React Router DOM
- State Management using React Hooks
- Responsive UI Design
- Form Validation
- Dynamic Content Rendering

Benefits

- Developed interactive user interfaces.
 - Improved user experience design.
 - Better understanding of responsive web applications.
-

Backend Development Skills

Developing backend services provided experience in server-side programming and API development.

Skills Learned

- Node.js
 - Express.js
 - RESTful API Design
 - CRUD Operations
 - Request and Response Handling
 - JSON Data Management
-

Benefits

- Improved backend programming skills.
 - Better understanding of client-server architecture.
 - Efficient API development experience.
-

Database Management Skills

The project improved understanding of data storage and retrieval using a JSON-based database.

Skills Learned

- Database Design
- JSON Data Storage
- CRUD Operations
- Data Validation
- Data Organization

Benefits

- Improved database management concepts.
 - Better understanding of structured data handling.
-

Software Testing Skills

Testing the application improved software quality assurance and debugging abilities.

Skills Learned

- API Testing using Postman
- Integration Testing
- Functional Testing
- Debugging
- Error Handling
- Browser Compatibility Testing

Benefits

- Improved software testing knowledge.
 - Ability to identify and resolve application errors.
-

- Better understanding of quality assurance processes.

Project Management Skills

The project followed a systematic software development process from planning to deployment.

Skills Learned

- Requirement Analysis
- System Design
- Module-wise Development
- Documentation
- Time Management
- Software Development Life Cycle (SDLC)

Benefits

- Improved project planning abilities.
- Better understanding of software engineering practices.
- Enhanced documentation skills.

Problem-Solving Skills

Throughout development, several implementation challenges were identified and resolved.

Skills Learned

- Logical Thinking
- Debugging Techniques
- Error Analysis
- Performance Optimization
- Solution Design

Benefits

- Increased confidence in solving technical problems.
- Improved analytical and decision-making skills.

Summary of Skills Acquired

The major skills gained during the development of the project include:

- React.js Development
- JavaScript Programming
- HTML5 and CSS3
- Node.js Development
- Express.js Framework
- RESTful API Development
- JSON Database Management
- React Router DOM
- Responsive Web Design
- State Management using React Hooks
- API Integration
- Software Testing
- Debugging and Troubleshooting
- Visual Studio Code
- Postman API Testing
- Git and GitHub
- Software Documentation
- Project Management
- Problem-Solving Skills

Conclusion

The development of the **Zivora – Online Shopping Platform** significantly enhanced both technical and professional competencies. The project provided practical experience in full-stack web development, REST API integration, database management, responsive user interface design, software testing, debugging, and documentation. In addition to technical knowledge, the project improved project management, problem-solving, and analytical skills. These experiences have strengthened the ability to develop modern web applications and will be valuable for future academic projects, internships, and professional software development careers.

CHAPTER -13 REFERENCES

The development of the **Intelligent Navigation and Safety Platform** involved the use of various technologies, frameworks, libraries, documentation resources, and online references. These resources provided guidance for system design, implementation, testing, deployment, and navigation-related functionalities.

Books

- [1] Roger S. Pressman and Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th Edition, McGraw-Hill Education, 2020.
- [2] Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2015.
- [3] Jon Duckett, *JavaScript and JQuery: Interactive Front-End Web Development*, Wiley Publications, 2014.
- [4] David Flanagan, *JavaScript: The Definitive Guide*, 7th Edition, O'Reilly Media, 2020.

Online Documentation

- [5] ReactJS Official Documentation. Available: <https://react.dev>
- [6] Vite Official Documentation. Available: <https://vitejs.dev>
- [7] Leaflet Official Documentation. Available: <https://leafletjs.com>
- [8] OpenStreetMap Documentation. Available: <https://www.openstreetmap.org>
- [9] Mozilla Developer Network (MDN) Web Docs. Available: <https://developer.mozilla.org>
- [10] Geolocation API Documentation. Available: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

Development Tools

- [11] Visual Studio Code Documentation. Available: <https://code.visualstudio.com>
- [12] Git Documentation. Available: <https://git-scm.com/doc>
- [13] GitHub Documentation. Available: <https://docs.github.com>
- [14] Chrome Developer Tools Documentation. Available: <https://developer.chrome.com/docs/devtools>

Mapping and Navigation Resources

- [15] OpenStreetMap Foundation Resources. Available: <https://www.openstreetmap.org>
- [16] Leaflet Mapping Library Documentation. Available: <https://leafletjs.com/reference.html>
- [17] Browser Geolocation API Documentation. Available: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

Software and Technologies Used

- [18] ReactJS Frontend Framework.
- [19] JavaScript Programming Language.
- [20] HTML5 and CSS3.
- [21] Vite Development Environment.
- [22] Leaflet Interactive Mapping Library.
- [23] OpenStreetMap Mapping Services.

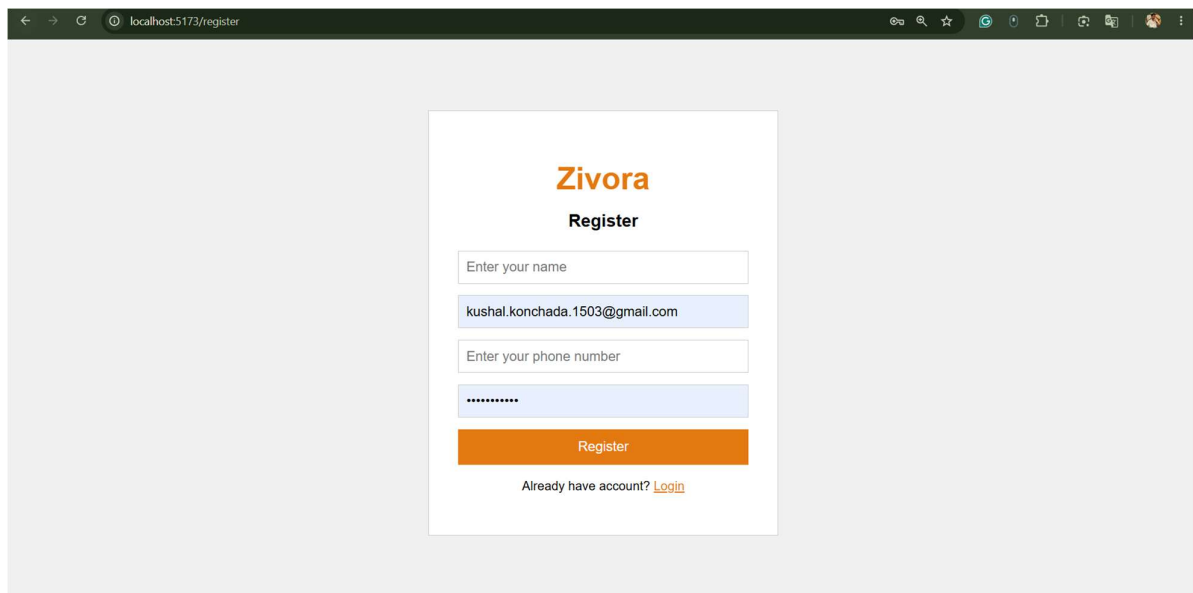
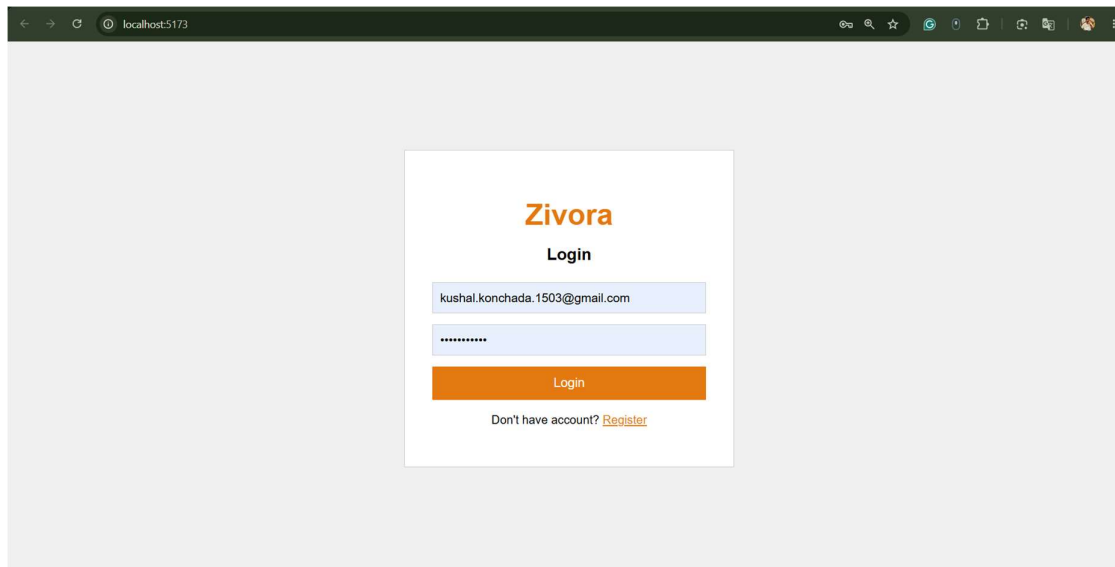
[24] Browser Geolocation API.

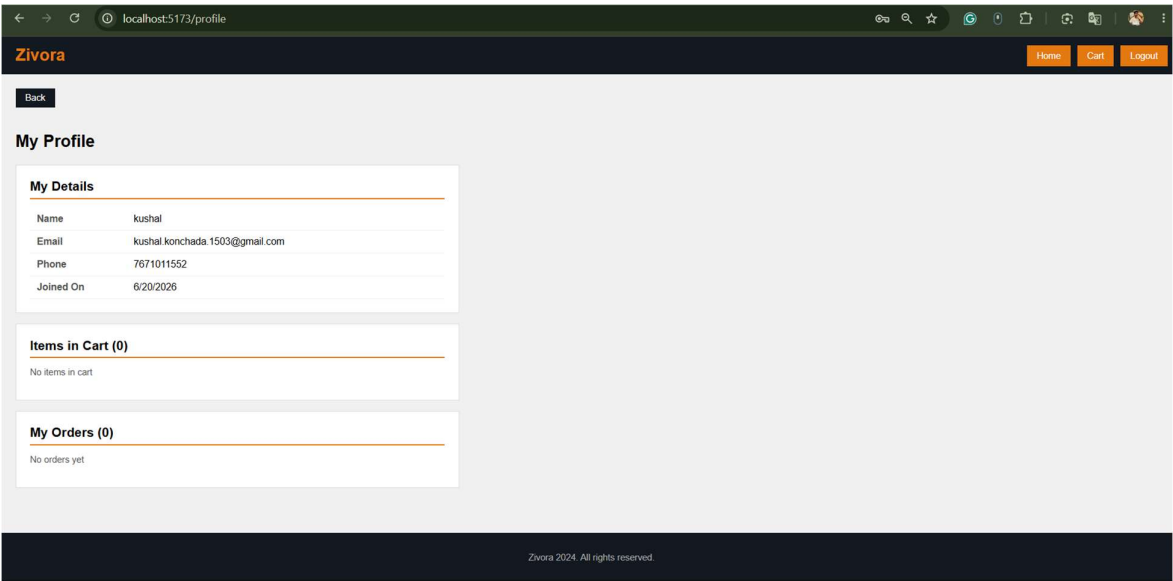
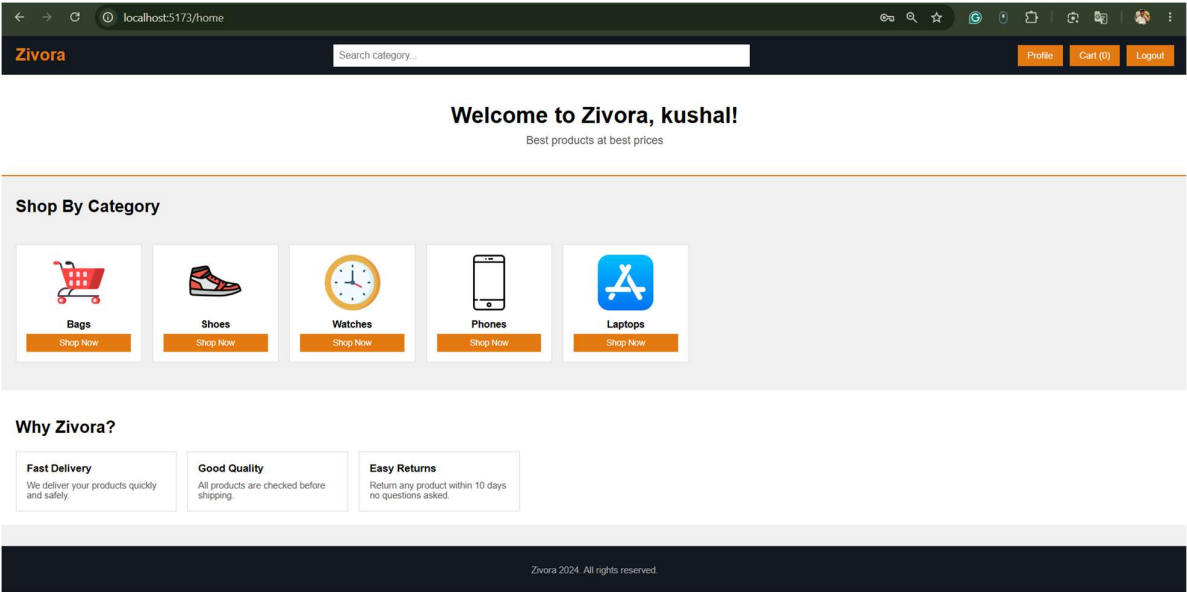
Conclusion

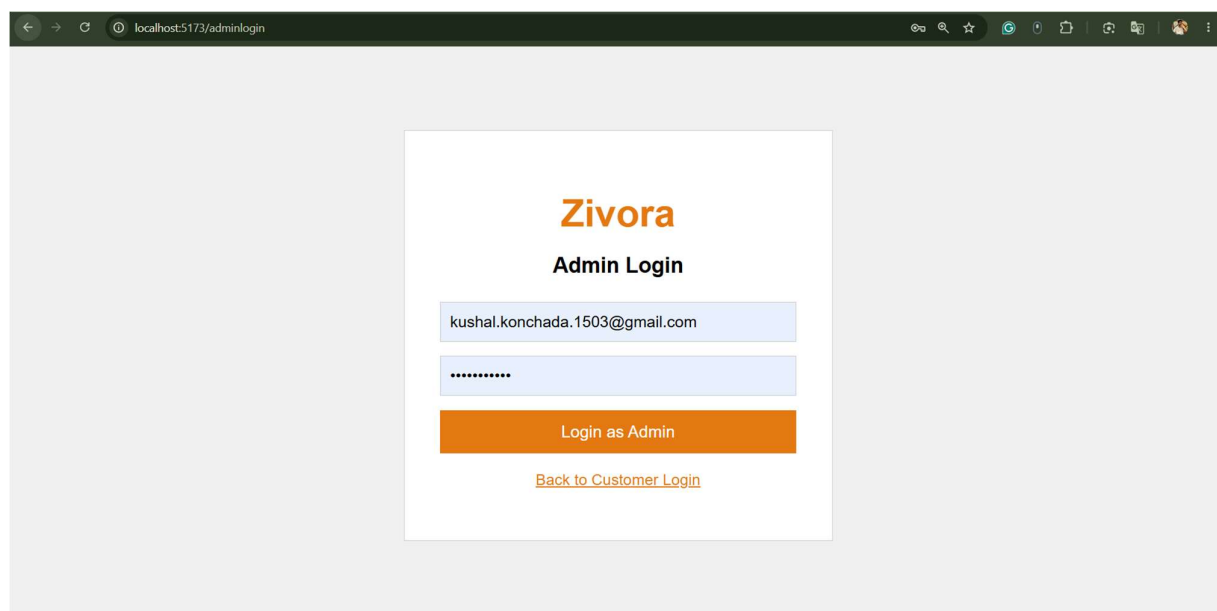
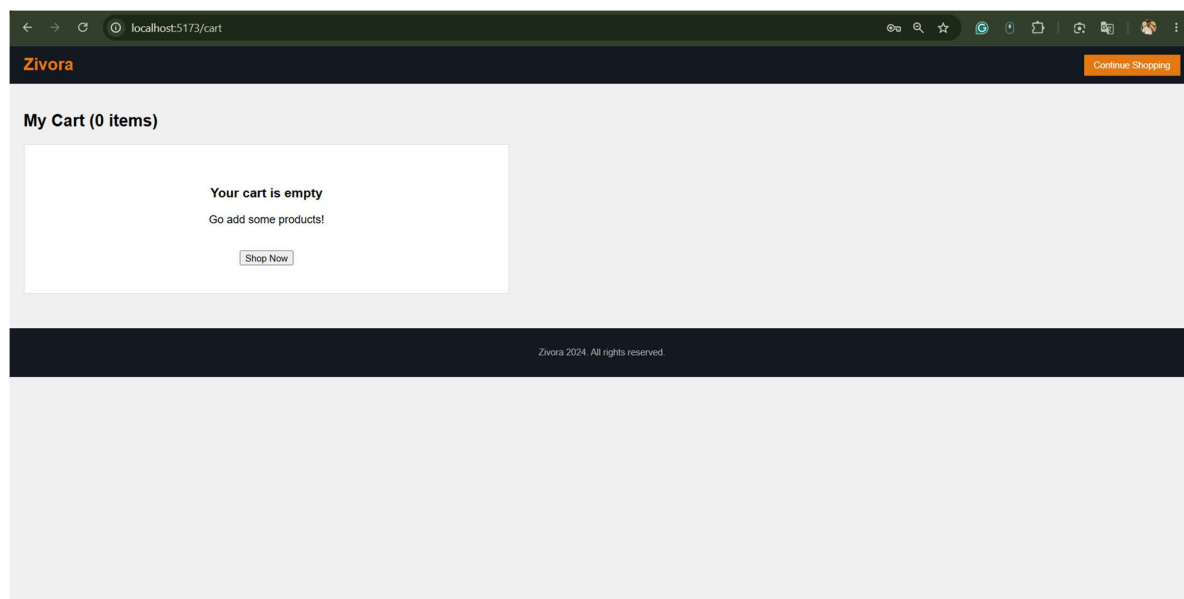
The above references provided the theoretical knowledge, technical guidance, development support, and implementation resources necessary for the successful development of the **Intelligent Navigation and Safety Platform**. These references were extensively used during system design, coding, testing, deployment, and documentation preparation.

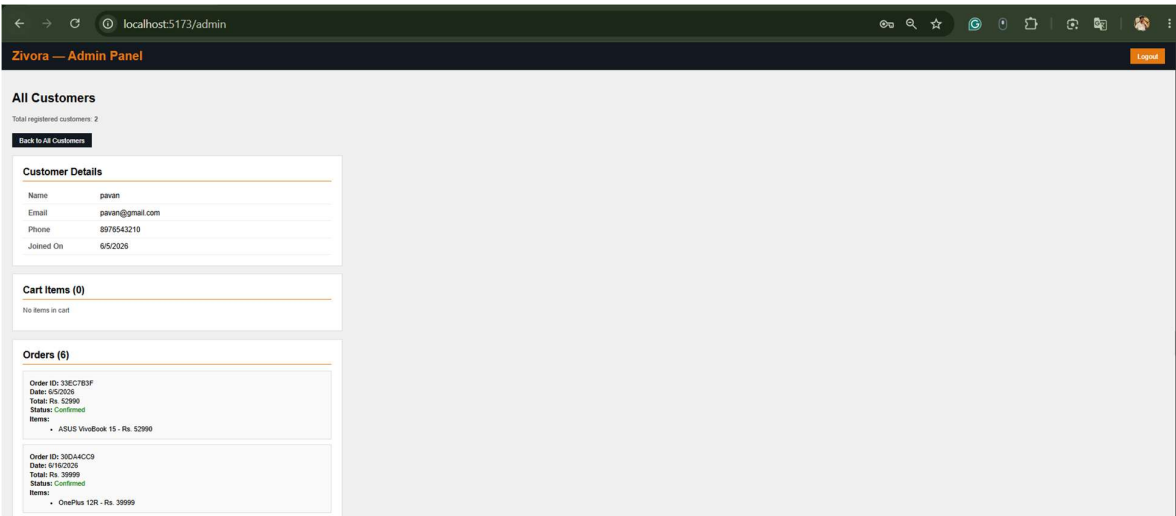
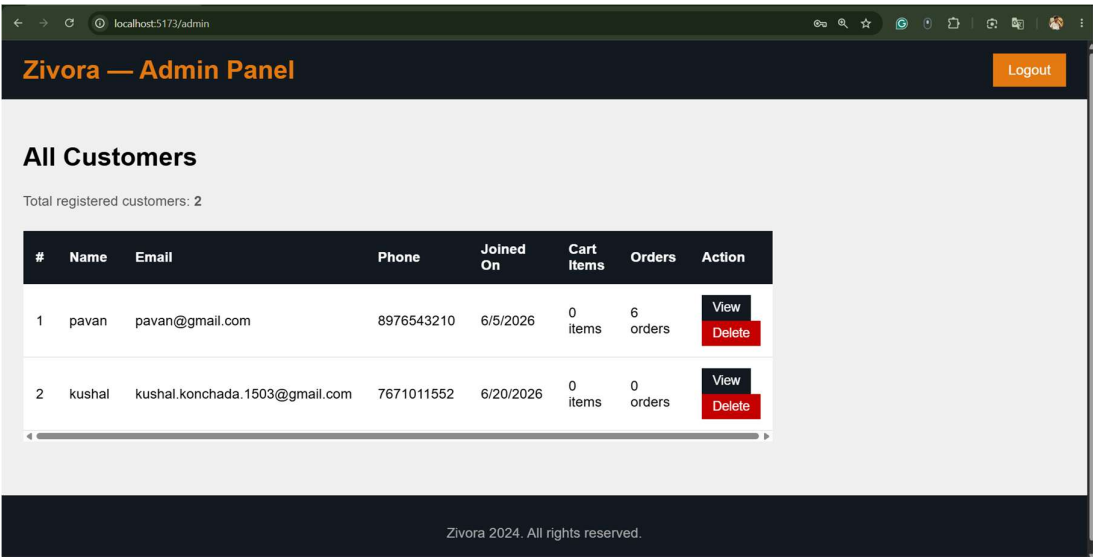
CHAPTER -14 APPENDICES

Screenshots of the Website

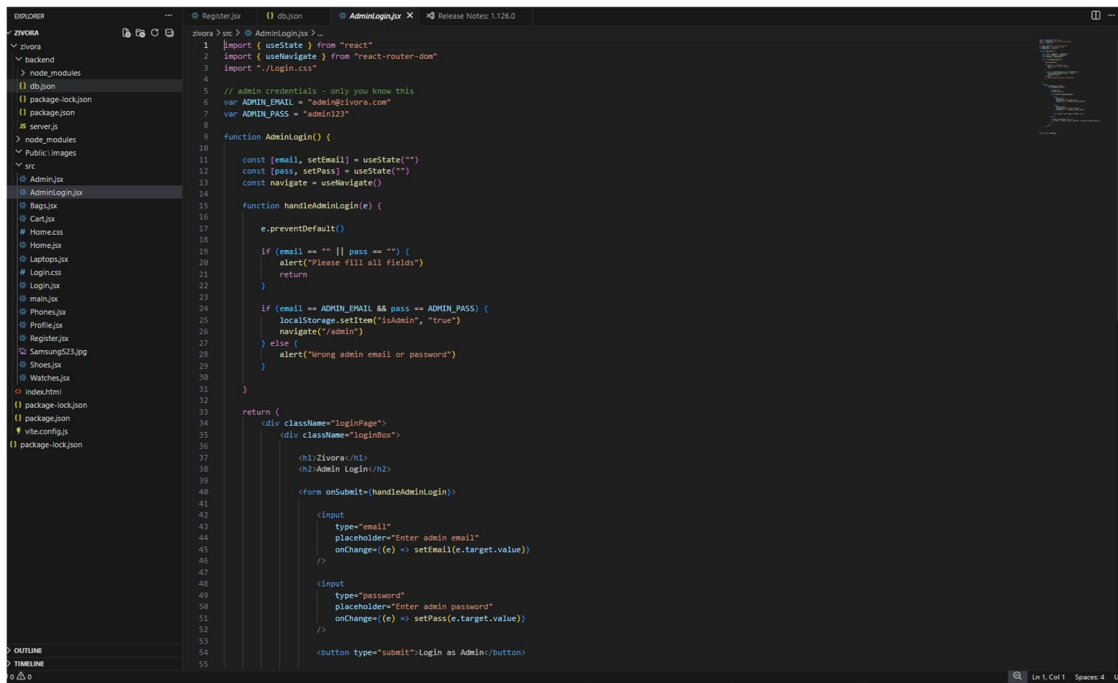




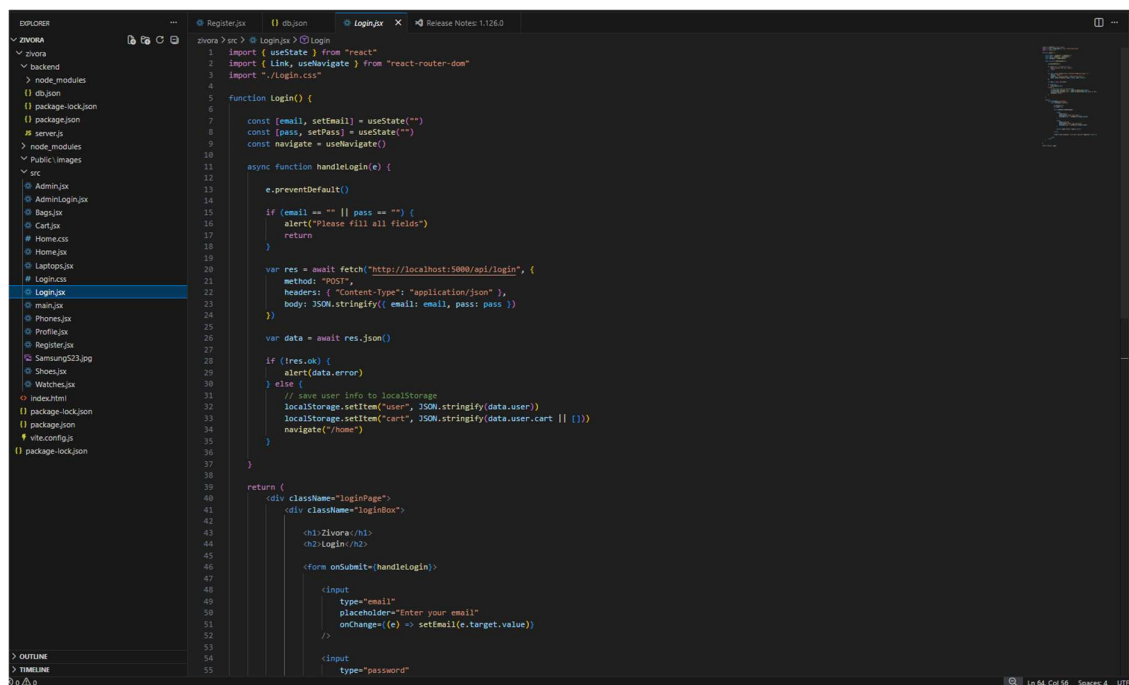




Sample code snippets



```
1 import { useState } from "react"
2 import { useNavigate } from "react-router-dom"
3 import "../Login.css"
4
5 // admin credentials - only you know this
6 var ADMIN_EMAIL = "admin@zivora.com"
7 var ADMIN_PASS = "admin123"
8
9 function AdminLogin() {
10
11   const [email, setEmail] = useState("")
12   const [pass, setPass] = useState("")
13   const navigate = useNavigate()
14
15   function handleAdminLogin(e) {
16
17     e.preventDefault()
18
19     if (email == "" || pass == "") {
20       alert("Please fill all fields")
21       return
22     }
23
24     if (email == ADMIN_EMAIL && pass == ADMIN_PASS) {
25       localStorage.setItem("isAdmin", "true")
26       navigate("/admin")
27     } else {
28       alert("Wrong admin email or password")
29     }
30   }
31
32   return (
33     <div className="loginPage">
34       <div className="loginBox">
35         <h1>Zivora</h1>
36         <h2>Admin Login</h2>
37         <form onSubmit={handleAdminLogin}>
38           <input
39             type="email"
40             placeholder="Enter admin email"
41             onChange={e => setEmail(e.target.value)}
42           />
43           <input
44             type="password"
45             placeholder="Enter admin password"
46             onChange={e => setPass(e.target.value)}
47           />
48           <button type="submit">Login as Admin</button>
49         </form>
50       </div>
51     </div>
52   )
53 }
```



```
1 import { useState } from "react"
2 import { Link, useNavigate } from "react-router-dom"
3 import "../Login.css"
4
5 function Login() {
6
7   const [email, setEmail] = useState("")
8   const [pass, setPass] = useState("")
9   const navigate = useNavigate()
10
11   async function handleLogin(e) {
12
13     e.preventDefault()
14
15     if (email == "" || pass == "") {
16       alert("Please fill all fields")
17       return
18     }
19
20     var res = await fetch("http://localhost:5000/api/login", {
21       method: "POST",
22       headers: { "Content-Type": "application/json" },
23       body: JSON.stringify({ email: email, pass: pass })
24     })
25
26     var data = await res.json()
27
28     if (!res.ok) {
29       alert(data.error)
30     } else {
31       // save user info to localStorage
32       localStorage.setItem("user", JSON.stringify(data.user))
33       localStorage.setItem("cart", JSON.stringify(data.user.cart || []))
34       navigate("/home")
35     }
36   }
37
38   return (
39     <div className="loginPage">
40       <div className="loginBox">
41         <h1>Zivora</h1>
42         <h2>Login</h2>
43         <form onSubmit={handleLogin}>
44           <input
45             type="email"
46             placeholder="Enter your email"
47             onChange={e => setEmail(e.target.value)}
48           />
49           <input
50             type="password"
51           />
52           <button type="submit">Login</button>
53         </form>
54       </div>
55     </div>
56   )
57 }
```

```
EXPLORER
  zivora
    backend
      node_modules
      db.json
      package-lock.json
      package.json
      server.js
      node_modules
      Public Images
      src
        Admin.jsx
        AdminLogin.jsx
        Bag.jsx
        Cart.jsx
        Home.css
        Home.jsx
        Laptops.jsx
        Login.css
        Login.jsx
        main.jsx
        Phones.jsx
        Profile.jsx
        Register.jsx
        SamsungS23.jpg
        Shoes.jsx
        Watches.jsx
        index.html
      package-lock.json
      package.json
      vite.config.js
      package-lock.json

  Register.jsx
  db.json
  Cart.jsx
  Release Notes: 1.126.0

zivora > src > Cart.jsx
1 import { useState } from "react"
2 import { useNavigate } from "react-router-dom"
3 import "../Home.css"
4
5 function Cart() {
6
7   const navigate = useNavigate()
8   const [cart, setCart] = useState(JSON.parse(localStorage.getItem("cart") || "[]"))
9
10  var user = JSON.parse(localStorage.getItem("user"))
11
12  function removeItem(index) {
13    var updated = cart.filter((item, i) => i !== index)
14    localStorage.setItem("cart", JSON.stringify(updated))
15    setCart(updated)
16  }
17
18  async function placeOrder() {
19
20    var total = 0
21    for (var i = 0; i < cart.length; i++) {
22      total = total + cart[i].price
23    }
24
25    if (user) {
26      // save order to backend
27      var res = await fetch("http://localhost:5000/api/placeorder/" + user.id, {
28        method: "POST",
29        headers: { "Content-Type": "application/json" },
30        body: JSON.stringify({ items: cart, total: total })
31      })
32      var data = await res.json()
33      if (res.ok) {
34        alert("Order placed! Order ID: " + data.order.orderId)
35      }
36
37      localStorage.setItem("cart", "[]")
38      setCart([])
39      navigate("/home")
40    }
41  }
42
43  var total = 0
44  for (var i = 0; i < cart.length; i++) {
45    total = total + cart[i].price
46  }
47
48  return (
49    <div>
50      <div className="navBar">
51        <a onClick={() => navigate("/home")}>Zivora</a>
52        <div className="navRight">
53          <button onClick={() => navigate("/home")}>Continue Shopping</button>
54        </div>
55      </div>
```

```
EXPLORER
  zivora
    backend
      node_modules
      db.json
      package-lock.json
      package.json
      server.js
      node_modules
      Public Images
      src
        Admin.jsx
        AdminLogin.jsx
        Bag.jsx
        Cart.jsx
        Home.css
        Home.jsx
        Laptops.jsx
        Login.css
        Login.jsx
        main.jsx
        Phones.jsx
        Profile.jsx
        Register.jsx
        SamsungS23.jpg
        Shoes.jsx
        Watches.jsx
        index.html
      package-lock.json
      package.json
      vite.config.js
      package-lock.json

  Register.jsx
  db.json
  Profile.jsx
  Release Notes: 1.126.0

zivora > src > Profile.jsx > Profile > useEffect callback
1 import { useState, useEffect } from "react"
2 import { useNavigate } from "react-router-dom"
3 import "../Home.css"
4
5 function Profile() {
6
7   const navigate = useNavigate()
8   const [profile, setProfile] = useState(null)
9
10  var user = JSON.parse(localStorage.getItem("user"))
11
12  useEffect(() => {
13    if (user) {
14      navigate("/")
15      return
16    }
17
18    fetch("http://localhost:5000/api/profile/" + user.id)
19      .then(res => res.json())
20      .then(data => setProfile(data))
21      .catch(err => alert("Cannot connect to server"))
22  }, [])
23
24  function logout() {
25    localStorage.removeItem("user")
26    localStorage.removeItem("cart")
27    navigate("/")
28  }
29
30  return (
31    <div>
32      <div className="navBar">
33        <a onClick={() => navigate("/home")}>Zivora</a>
34        <div className="navRight">
35          <button onClick={() => navigate("/home")}>Home</button>
36          <button onClick={() => navigate("/cart")}>Cart</button>
37          <button onClick={logout}>Logout</button>
38        </div>
39      </div>
40      <div className="cartPage">
41        <button className="backBtn" onClick={() => navigate("/home")}>Back</button>
42      </div>
43      <h2>My Profile</h2>
44      {profile && <p>Loading...</p>}
45      {profile && (
46        <div>
47          <div className="profileBox">
```

```
EXPLORER
vzovra
  backend
    node_modules
    db.json
    package-lock.json
    package.json
    server.js
    node_modules
    public/images
    src
      Admin.jsx
      AdminLogin.jsx
      Bag.jsx
      Cart.jsx
      Home.css
      Home.jsx
      Laptops.jsx
      Login.css
      Login.jsx
      main.jsx
      Phones.jsx
      Profile.jsx
      Register.jsx
      SamsungS23.jpg
      Shoes.jsx
      Watches.jsx
      index.html
    package-lock.json
    package.json
    vite.config.js
    package-lock.json

vzovra > backend > db.json > {} customers > {} > {} orders > {} 0
1 {
2   "customers": [
3     {
4       "id": "4a62ccc1-83b7-4226-ab45-cfb027966648",
5       "name": "pavan",
6       "email": "pavan@gmail.com",
7       "phone": "8078543210",
8       "pass": "asdfghjkl;",
9       "cart": [],
10      "orders": [
11        {
12          "orderId": "33EC783F",
13          "items": [
14            {
15              "id": 1,
16              "name": "ASUS VivoBook 15",
17              "price": 52990,
18              "image": "https://cdn-icons-png.flaticon.com/512/5977/5977575.png"
19            }
20          ],
21          "total": 52990,
22          "date": "6/5/2026",
23          "status": "Confirmed"
24        },
25        {
26          "orderId": "380A4CC9",
27          "items": [
28            {
29              "id": 3,
30              "name": "OnePlus 12R",
31              "price": 39999,
32              "image": "https://cdn-icons-png.flaticon.com/512/545/545245.png"
33            }
34          ],
35          "total": 39999,
36          "date": "6/16/2026",
37          "status": "Confirmed"
38        },
39        {
40          "orderId": "E289D620",
41          "items": [
42            {
43              "id": 2,
44              "name": "HP Pavilion 14",
45              "price": 64990,
46              "image": "https://cdn-icons-png.flaticon.com/512/5977/5977575.png"
47            }
48          ],
49          "total": 64990,
50          "date": "6/16/2026",
51          "status": "Confirmed"
52        }
53      ],
54      "orderId": "5AEECF7D",
```

Installation / Setup Instructions

The Zivora – Online Shopping Platform is developed using React.js, Vite, Node.js, and Express.js. The application can be installed and executed on any computer with Node.js, npm, and a modern web browser.

Steps:

1. Download or clone the project source code from the GitHub repository.
2. Open the project folder in Visual Studio Code.
3. Open the integrated terminal and navigate to the project directory.
4. Install all required dependencies by running:
`npm install`
5. Navigate to the backend folder and start the backend server:
`npm start`
(or)
`node server.js`
6. Open a new terminal, navigate to the frontend folder, and start the React application:
`npm run dev`
7. Open a web browser and visit:
`http://localhost:5173`
8. The Zivora – Online Shopping Platform will launch successfully, allowing users to browse products, manage their shopping cart, place orders, and access administrator features.

User Manual / Guide

The Zivora – Online Shopping Platform provides customers with a simple and secure environment for online shopping. Users can browse products, manage their shopping cart, place orders, and update their profile information through an intuitive and responsive interface.

Steps to Use:

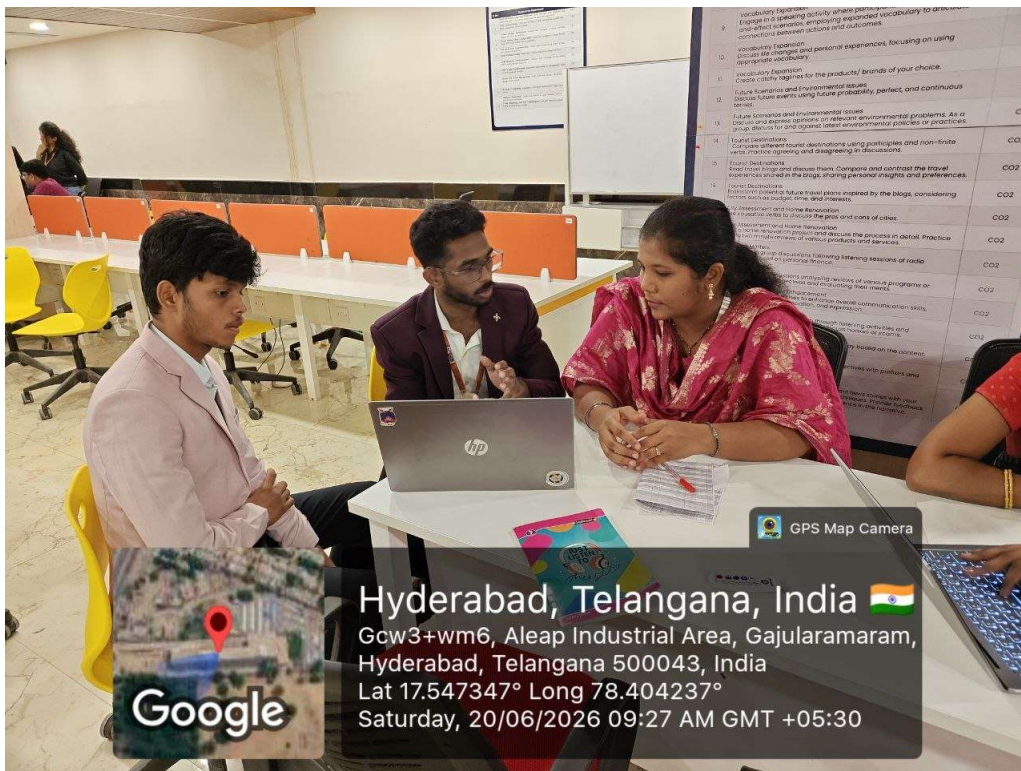
1. Open the application in a web browser.
2. Register a new account or log in using valid credentials.
3. Browse products from different categories such as Phones, Laptops, Watches, Bags, and Shoes.
4. Use the search feature to quickly find the required product.
5. Click on a product to view its details, price, and description.
6. Add desired products to the Shopping Cart.
7. Review the shopping cart and update product quantities if required.
8. Proceed to place the order and confirm the purchase.
9. Visit the Profile page to view or update personal information.

10. Administrators can log in to access the Admin Dashboard, where they can manage products, categories, users, and customer orders.
-

System Requirements

- Operating System: Windows 10/11, Linux, or macOS
- Node.js: Version 18 or later
- npm: Latest version
- Code Editor: Visual Studio Code
- Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge
- Internet Connection: Required for downloading dependencies

Geo Tag photos with guide



Poster:



Zivora — Online Shopping Platform

React · Vite · Express · React Router · REST API · JSON Storage

PROJECT BASED LEARNING - FEDF

TEAM

- K. Kushal Kumar
2520040031
- T. Sai Pavan Reddy
2520040018

- React 19 + Vite
- Express.js Backend
- React Router DOM
- React Hooks
- JSON File Storage
- SPA Architecture

- Admin Dashboard
- Cart & Orders

ABSTRACT

Zivora is a full-stack web-based Online Shopping Platform built with **React + Vite** (frontend) and **Express.js** (backend). It lets customers browse products across **Phones, Laptops, Watches, Bags, and Shoes**, manage a shopping cart, and place orders — all through a clean, responsive SPA.

- Secure customer **Register / Login** with session stored in localStorage.
- Per-customer **cart persistence** synced to the Express backend via REST APIs.
- Full **order placement** flow — cart is cleared and order saved on checkout.
- Dedicated **Admin module** — view and delete registered customers.
- Uses **React Hooks** (useState, useEffect) for all dynamic state management.
- Component-based architecture** with reusable pages and protected routes.

COURSE OUTCOME MAPPING

CO1

React Fundamentals

- Component-based architecture
- JSX for UI rendering
- Reusable page components
- Login, Register, Home, Cart

CO2

JavaScript & ES6

- Arrow functions throughout
- Event handling & form submit
- Array .map() for product lists
- Destructuring & modules

CO3

Props & State

- useState for cart, forms
- Props passing to children
- Controlled input components
- Login, Register, Order forms

CO4

Hooks & Lifecycle

- useEffect for data fetching
- Profile & order history load
- Dynamic content rendering
- Dashboard & category pages

CO5

SPA Routing

- React Router DOM v7
- BrowserRouter + Routes
- Protected & admin routes
- Login → Home → Cart → Profile

CO6

Full-stack Deployment

- Vite-based project setup
- Express REST API backend
- Responsive CSS design
- Integrated module structure

SYSTEM ARCHITECTURE

```
graph TD; A[Customer / Admin  
Browser Interaction] --> B[React Frontend (Vite)  
Components · Router · Hooks]; B --> C[REST API Calls  
fetch() to Express :5000]; C --> D[Express.js Backend  
Routes · Middleware · Logic]; D --> E[db.json Storage  
Customers · Carts · Orders]; E --> F[Admin Dashboard  
View · Manage · Delete Users];
```

STUDENT DETAILS

STUDENT 1

K. Kushal Kumar
Roll No: 2520040031

STUDENT 2

T. Sai Pavan Reddy
Roll No: 2520040018

DEPARTMENT

ECE
B.Tech – I Year

SUBJECT

FEDF
Project Based Learning

CONCLUSION



Full Shopping Flow

Zivora delivers a complete end-to-end e-commerce experience — browse, cart, checkout — with **real backend persistence** per customer.



Modern React Architecture

Built on **component-based SPA** principles using React Hooks and Router DOM, demonstrating scalable front-end engineering patterns.



Future-Ready Foundation

Structured for extensions like **real-time payments, live tracking, analytics**, and cloud DB — replacing the JSON file with minimal changes.

KLH (Deemed to be University) · Department of ECE · Academic Year 2024–25

Zivora — Online Shopping Platform